

LOG430

Architecture logicielle — Été 2026

Phase 2 — Architecture orientée événements (event-driven — saga chorégraphiée)

Dossier d'architecture — delta événementiel · gabarit Arc42 + vues 4+1 + ADR
Projet CanTelcoX — Système de support commercial (BSS) télécom

Champ	Contenu
Équipe / N° d'équipe	Équipe A
Membres (nom, code permanent)	Derek Lefebvre-Dupras, LEFD26029807 Miriam Brillon-Emond, BRIM23619601 Vincent de Grandpré, DEGV03078209 Billy Taing, TAIB73290301 Eric Kuité, KUIE28328902
Cours / Groupe	LOG430 — Groupe 02
Session	Été 2026
Date de remise	5 août 2026, 23 h 59 (Moodle)
Phase précédente (référence)	Dossier Phase 1 — Architecture basée par services (microservices)
Version du document	v1.0 - 2026-08-05

École de technologie supérieure — Département de génie logiciel et des TI



ÉCOLE DE
TECHNOLOGIE
SUPÉRIEURE
Université du Québec



Sommaire

Sommaire.....	2
1. Introduction et objectifs.....	5
1.1 Rappel du contexte & état à l'issue de la Phase 1.....	5
1.2 Objectifs de la Phase 2.....	5
1.3 Complétion des récits utilisateur (user stories).....	5
1.4 Reprise des items manquants de la Phase 1.....	6
1.5 Objectifs de qualité.....	7
1.6 Parties prenantes.....	8
2. Contraintes d'architecture.....	9
2.1 Contraintes techniques (delta).....	9
2.2 Contraintes de cohérence & de livraison.....	9
3. Contexte et périmètre — cartographie des événements.....	10
3.1 Périmètre du changement.....	10
3.2 Découverte des événements (event storming).....	10
3.3 Événements de domaine — détail par agrégat.....	11
3.4 Catalogue d'événements (vue d'ensemble).....	13
4. Stratégie de solution — passage à l'événementiel.....	14
4.1 Style architectural événementiel (chorégraphie).....	14
4.2 Broker de messages.....	17
4.3 Outbox pattern.....	17
4.4 Saga chorégraphiée & compensation.....	18
4.5 Cohabitation REST synchrone ↔ événementiel.....	19
4.6 Intégration des systèmes fournis (HLR/HSS & PortabilityHub) — obligatoire.....	20
5. Vue des blocs de construction.....	22
5.1 Vue d'ensemble : services + broker.....	22
5.2 Composants événementiels d'un service.....	25
5.3 Modèle & versionnement des schémas d'événements.....	25
5.4 Enveloppe commune des messages.....	26
6. Vue d'exécution — flux événementiels.....	27
6.1 Saga chorégraphiée — scénario nominal.....	27
6.2 Compensation (échec d'une étape).....	30
6.3 Fiabilité outbox → publication → consommation.....	32
Persistance des données dans l'Outbox.....	32
6.4 Portabilité via le PortabilityHub fourni (port-in).....	33
7. Vue de déploiement.....	36
7.1 Topologie de déploiement (delta).....	36

Réseaux.....	38
Volumes.....	38
État de santé.....	38
7.2 Tracing distribué.....	39
7.3 API Gateway & load balancing.....	41
Exposition du statut d'un traitement asynchrone.....	41
Propagation du contexte.....	42
Répartition de charge.....	42
Frontière entre la Gateway et le système événementiel.....	43
Ajustements apportés en Phase 2.....	43
8. Concepts transversaux.....	45
8.1 Outbox pattern.....	45
8.2 Saga chorégraphiée & compensation.....	45
8.3 Idempotence des consommateurs & déduplication.....	45
8.4 Ordre & garanties de livraison.....	46
8.5 Cohérence à terme (eventual consistency).....	46
8.6 Observabilité événementielle (4 Golden Signals + tracing).....	46
8.7 Gestion d'erreurs événementielle (DLQ, retries, backoff).....	47
8.8 Versionnement des schémas d'événements.....	47
8.9 Intégration fiable des systèmes fournis (HLR/HSS & PortabilityHub).....	47
9. Décisions d'architecture (ADR).....	49
ADR-004 — Adoption de l'architecture événementielle (chorégraphie).....	49
ADR-005 — Outbox pattern pour la publication fiable.....	49
ADR-006 — Saga chorégraphiée & compensation.....	50
ADR-007 — Choix du broker de messages.....	51
ADR-008 — Intégration des systèmes fournis (HLR/HSS & PortabilityHub) via adapters/ACL.....	51
ADR-009 — Utilisation de AsyncAPI.....	52
ADR-010 — Garantie de publication des messages et cohérence des noeuds.....	53
ADR-011 — Cohabitation REST synchrone ↔ événementiel.....	53
10. Exigences de qualité — gains mesurés.....	55
10.1 Cibles NFR & résultats.....	55
10.2 Résultats des campagnes de charge.....	55
10.3 Comparatif synchrone REST ↔ événementiel.....	55
10.4 Observabilité : tracing, lag broker & 4 Golden Signals.....	56
10.5 Load balancing & caching.....	56
11. Risques et dette technique.....	57
12. Glossaire.....	58
12.1 Termes événementiels.....	58

12.2 Acronymes	58
12.3 Termes métier	59
Annexe A — Catalogue d'événements & schémas	62
Annexe B — Matrice des récits utilisateur (couverture complète).....	69
Annexe C — Traçabilité, Définition de Fini & auto-évaluation	70
C.1 Matrice de traçabilité (livrables & critères d'acceptation).....	70
C.2 Définition de Fini (DoD) — Phase 2.....	71
C.3 Auto-évaluation.....	71
Annexe D — Soutenance et démonstration	72
D.1 Déroulé attendu	72
D.2 Scénario de démonstration.....	72
Annexe E - AsyncAPI.....	73

1. Introduction et objectifs

Objectif Arc42 — Présenter la Phase 2 : la transition vers l'événementiel, l'achèvement des récits utilisateur et des items manquants de la Phase 1, et les objectifs de qualité (gains mesurés de latence, de débit et d'observabilité).

1.1 Rappel du contexte & état à l'issue de la Phase 1

CanTelcoX, opérateur mobile canadien fictif, exploite un BSS couvrant le cycle de vie des lignes mobiles (souscription, activation, consultation d'usage, commande de forfaits/options, paiement, conformité CRTC/Loi 25). La Phase 1 a livré une architecture basée par services (microservices synchrones REST derrière une API Gateway).

Or, les limites de l'architecture synchrone ont été atteintes assez rapidement et même s'il n'y pas de saturation du système, dans la majorité des cas, on veut basculer vers une architecture événementielle pour optimiser la performance du système.

Tous les cas d'utilisation ont été couverts dans la phase I, certains attendaient la phase II pour être implémentés.

1.2 Objectifs de la Phase 2

La Phase 2 vise à découpler les services par des échanges asynchrones d'événements, à garantir la cohérence des transactions distribuées sans coordinateur central (saga chorégraphiée) et à fiabiliser la publication via l'outbox pattern.

Objectif Phase 2	Motivation	Cible / résultat attendu
Découplage événementiel	Réduire le couplage temporel entre services	Chaque service agit selon des événements émis par d'autres
Saga chorégraphiée	Cohérence des transactions distribuées sans orchestrateur	Saga de commande et activation de ligne mobile chorégraphiée et implémentée
Outbox pattern	Publication atomique avec l'écriture métier	Tous les messages émis sont traçable dans une table Outbox. Aucune perte de message et aucun doublon.
Gains de performance / résilience	Absorber les pics, isoler les pannes	Gains de performance significatifs en latence et recouvrement lors de panne.
Observabilité de bout en bout	Tracer un flux à travers les événements	Tracing distribué actif et granulaire.

1.3 Complétion des récits utilisateur (user stories)

La Phase 2 exige la couverture de l'ensemble des récits utilisateur du cahier (au-delà des ≥ 5 UC Must de la Phase 1). Renseignez le backlog complet en Annexe B et détaillez ci-dessous les récits ajoutés en Phase 2.

UC Phase 1 (manquant/partiel)	Critère d'acceptation	Statut
-------------------------------	-----------------------	--------

UC-01 - Inscription & vérification d'identité	Il faut ajouter la preuve d'adresse qui est un critères réglementaire	Partiel
UC-02 - Authentification & MFA (OTP / WebAuthn)	OTP seul. pour opération sensible à implémenter	Partiel
UC-03 - Activation d'une ligne mobile (MSISDN / portabilité)	Une nouvelle ligne est activée sur le réseau et il est possible de porter un numéro d'un autre BSS	Partiel
UC-04 - Consultation de l'usage et des factures	Gestion et consultation des usages à implémenter	Partiel
UC-07 - Détection de fraude (SIM swap, usage anormal, roaming)	Une vérification antifraude est faite sur activation de ligne et sur inscription	À faire
UC-08 - Cycle de facturation mensuel	Les forfaits mensuels et les frais d'usages sont facturés en lots	À faire

1.4 Reprise des items manquants de la Phase 1

Recensez les livrables Phase 1 non réalisés (ou partiels) et repris en Phase 2.

Item Phase 1 (manquant/partiel)	Action de reprise	Preuve (section / dépôt)
Répartition de charge	Compléter la section 7.3 de la documentation d'architecture PI : plan de test N = 1..4 et terminaison d'instance.	Implémenté dans tous les services, voir <i>docker-compose.yml</i> de chacun, et dans la passerelle api.
Idempotence à l'activation de ligne	Compléter la section 8.3 de la documentation d'architecture : idempotence à l'activation de ligne, portée à implémenter et documenter.	Voir service client, une même ligne ne peut pas être activée deux fois.
Garantie d'écriture unique	Compléter la section 8.3 de la documentation d'architecture : Garantie d'écriture unique par déduplication et patron Outbox	<i>Exactly-once</i> garanti par patron Outbox du producteur + patron d'idempotence du consommateur.
Journal immuable	Compléter section 8.4 de la documentation d'architecture : Structure du journal immuable, événements consignés et	Garanti par le broken Kafka et les fichiers persistés sur son disque dur. Le journal immuable est extrait périodiquement

	garantie d'immutabilité.	pour sauvegarde externe à perpétuité.
Détection antifraude	Compléter section 8.5 de la documentation d'architecture : détection SIM Swap, usurpation d'identité, usage anormal, règles et tests associés.	Service antifraude en place mais qui ne fait que recevoir des messages sans plus.
Antémémoire (caching)	Compléter la section 8.9 de la documentation d'architecture : cache sur catalogue et usages, TTL, invalidation, risques de péremption et gains chiffrée en 10.7.	Pas de changement.
Test de couverture	Compléter la section 10.4 de la documentation d'architecture : tests de couverture sur plus de 80 % du domaine critique.	Pas de changement.
OTP sur opération sensible	Ajouter le traitement d'un code OTP pour sécuriser la commande finalisation d'une commande.	Pas de changement.
Activation d'une ligne mobile	Implémentation de UC-05	Implémenté dans svc-lignes (api-usage) et fonctionnel bout-en-bout.
Produire les factures selon les usages	Implémentation de UC-08	Pas de changement.
Consultation des usages	Les rapports d'usage sont persistés dans la base de données de facturation et consultables par Postman	Via api-usage.

1.5 Objectifs de qualité

Les objectifs de qualité prioritaires de la Phase 2 (cibles NFR reconduites de la Phase 1, enrichies des propriétés événementielles) :

Objectif de qualité	Scénario / motivation	Cible mesurable
Performance	Latence de bout en bout des flux clés	P95 ≤ 250 ms (500 ms en Phase 1) ; débit ≥ 1000 ops/s (600 en Phase 1)
Disponibilité / résilience	Panne d'un consommateur sans perte d'événement	Dispo 99 % (95 % en Phase 1) ; reprise sans perte

Cohérence (eventual consistency)	Convergence après saga / compensation	[À compléter — délai de convergence cible]
Auditabilité / traçabilité	Suivre un flux via tracing + journal	[À compléter — trace complète bout-en-bout]

1.6 Parties prenantes

Partie prenante	Rôle / attente	Section concernée
Responsable du produit	Priorisation des tâches et réalisation du système	§1.3 ; Annexe B
Équipe d'architecture	Décisions, choix architecturaux et implémentation de base	§4 ; §9
Exploitation / SRE	Observabilité, traçabilité, métriques et état de la messagerie distribuée	§7 ; §8.6
Conformité (CRTC/Loi 25)	Audit des transactions, règlement d'identification et détection antifraude	§8

2. Contraintes d'architecture

Objectif Arc42 — Recenser les contraintes propres à la transition événementielle qui s'ajoutent aux contraintes de la Phase 1.

2.1 Contraintes techniques (delta)

Contrainte	Description / impact
Broker de messages	Kafka ; hébergé avec le registre de schémas Confluent via Docker Compose
Schémas d'événements	Les schémas d'événement sont au format JSON qui sont plus facile à lire pour un humain. Ce choix pourra être revu pour utiliser un schéma binaire comme Avro pour un gain de performance. Le versionnement se fait avec un registre de schémas Confluent.
Tracing distribué	Traçabilité des événements dans leur contexte via OpenTelemetry + Jaeger
HLR / HSS fourni (OBLIGATOIRE)	Provisionnement et activation des lignes via l'adapter du HLR/HSS fourni — pas de simulation maison ; l'appel doit être idempotent et instrumenté (tracing)
PortabilityHub fourni (OBLIGATOIRE)	Port-in / port-out des numéros (MSISDN) via le hub de portabilité fourni — statut final PORTED / REJECTED / EXPIRED tracé ; intégration via adapter + idempotencyKey
Conservation Phase 1	REST synchrone, API Gateway, Prometheus/Grafana, LB et cache demeurent en place

Exigence : le HLR/HSS et le PortabilityHub sont des systèmes fournis par l'encadrement. Leur utilisation est obligatoire — ils ne doivent pas être réimplémentés. Ils sont intégrés via des adapters (ports/adapters + couche anticorruption) et participent aux sagas (voir §4.6 et §6).

2.2 Contraintes de cohérence & de livraison

Cadre	Exigence dérivée
Livraison des messages	at-least-once ; idempotence côté consommateur ; audit et traçabilité
Ordre des événements	corrélation des événements et partitionnement par clé d'agrégat
Exactly-once (effet)	Kafka avec exactly-once via clé d'idempotence et le patron Outbox pour un effet unique
Conformité CRTC / Loi 25	Journal d'audit append-only alimenté par les événements

3. Contexte et périmètre — cartographie des événements

Objectif Arc42 — Délimiter le périmètre du changement événementiel et cartographier les événements du domaine (event storming) ainsi que leurs producteurs/consommateurs.

3.1 Périmètre du changement

En Phase 1, les services communiquaient de façon synchrone (REST) : la prise de commande appelait successivement l'activation puis la facturation, ce qui crée un couplage temporel (une panne en aval bloque l'amont) et cumule les latences. En Phase 2, ces effets de bord sont propagés par des événements de domaine publiés sur le broker : chaque service réagit et enchaîne la saga sans appel synchrone bloquant.

Le périmètre du changement implique presque tous les services afin de l'architecture réactive et événementielle. Par *Saga chorégraphiée* il est entendu la saga de création de commande avec ou sans activation de ligne mobile avec ou sans portabilité.

Entité	Agrégat	Événements	Justification
CustomerBill	Facturation et usages	CUD	L'utilisateur reçoit des communications sur l'état de ses factures.
BillCycle	Facturation et usages	C	La création d'un cycle de facturation déclenche la création des factures.
Usage	Facturation et usages	C	La création d'un usage ajoute incrémente les compteurs d'utilisation.
Party	Clients	UD	La désactivation ou la suppression annule les commandes et libère les lignes activées et les numéros mobiles.
ProductOrder	Commandes	CUD	Saga chorégraphiée et compensation.
Product	Commandes	CUD	Saga chorégraphiée et compensatoir, un produit référant à un service déclenche l'activation d'une ligne.
Service	Lignes et activations	CUD	Saga chorégraphiée, ajout de ligne au profil du client, finalisation de commande et compensation.
AntiFraude	Audit	C	Saga chorégraphiée, circuit-breaker de processus.

[À compléter — précisez quels services deviennent producteurs/consommateurs, et quels flux synchrones de la Phase 1 sont remplacés ou doublés par des flux événementiels. Insérer au besoin un schéma « avant / après ».]

3.2 Découverte des événements (event storming)

Résultat de l'atelier event storming, lu de gauche à droite : une commande (intention) est traitée par un agrégat qui produit un événement de domaine (fait passé, immuable) ; une politique (réaction) observe cet événement et déclenche la commande suivante, tissant ainsi la chorégraphie. Les événements de compensation défont une étape en cas d'échec.

Événement	Agrégat / service	Événement de domaine (passé)	Politique / réaction (suivant)
SoumettreCommande	Chorégraphe	SAGA_SUBMITTED	→ préparer un brouillon de commande
CreerBrouillon	Commandes	ORDER_DRAFT	→ faire des vérifications
VerifierClient	Clients	BUYER_VERIFIED	→ vérification client OK
VerifierOffres	Catalogue	OFFERINGS_VERIFIED	→ vérification commande OK
VerifierService	Lignes	SERVICE_VERIFIED	→ vérification service OK
VerificationsToutesOK	Chorégraphe	ORDER_VALIDATIONS_OK	→ créer la commande
ValiderCommande	Commandes	ORDER_CREATED	→ gestion de stock
DiminuerStock	Catalogue	STOCK_DECREASED	→ activation de services
ActiverServices	Chorégraphe	DO_PROVISION_ACTIONS	→ portabilité et activation HLR
ProvisionnerLigne	Lignes	SERVICE_PROVISIONED	→ finaliser la commande
FinaliserCommande	Chorégraphe	DO_INVOICING	→ facturation et paiement
FacturePonctuelle CycleFacturesMensuel	Facturation	BILLCYCLE_CREATED	→ générer la facture → production en lot des factures mensuelles
FacturerCommande FacturerForfaitUsages	Facturation	CUSTOMERBILL_CREATED PAYMENT_CREATED	→ envoyer la facture → recevoir le paiement
ReceptionDePaiement	Facturation	PAYMENT_PROCESSED	→ Fin de la saga
DetectionDeFraude	Audit	FRAUD_DETECTED	→ Interrompre la saga
DetectionAbus	Audit	ABUS_USAGE	→ Informer le client
PorterNumero	Lignes	PORTABILITE_OK	→ Autoriser la portabilité du numéro

3.3 Événements de domaine — détail par agrégat

Pour chaque agrégat, précisez l'événement émis, l'invariant/la règle métier qui le déclenche et sa nature (domaine ou compensation). Les schémas de charge utile figurent en Annexe A.

Agrégat (bounded context)	Événement émis	Déclencheur / invariant métier	Nature
Audit	FRAUD_DETECTED	SIM Swap, usurpation d'identité	Domaine
	ABUS_USAGE	Abus d'utilisation	Domaine
Catalogue	OFFERINGS_VERIFIED	Vérification des offres	Domaine
	OFFERINGS_INVALID	La vérification des offres échoue	Compensation

	STOCK_DECREASED	Commander créée, à livrer	Domaine
	STOCK_INCREASED	Stock manquant pour remplir la commande	Compensation
Chorégraphe	SAGA_SUBMITTED	Commande soumise avec idempotency-key	Compensation
	ORDER_VALIDATIONS_OK	Toutes les validations de commandes sont faites et bonnes	Domaine
	ORDER_VALIDATIONS_KO	Erreurs de validation de commande	Compensation
	DO_PROVISION_ACTIONS	Inventaire mis à jour	Domaine
	DO_INVOICING	Les produits / services sont rendus	Domaine
	SAGA_ENDED	La commande est terminée	Domaine
Client	BUYER_VERIFIED	Vérification du client	Domaine
	BUYER_INVALID	La vérification du client échoue	Compensation
	PARTY_IDENTIFIED	Un usager s'est inscrit et a soumis sa pièce d'identité	Domaine
Commandes	ORDER_DRAFT	Préqualifier une commande	Domaine
	ORDER_CREATED	Commande qualifiée	Domaine
	ORDER_CANCELED	La commande est annulée	Compensation
Facturation	BILLCYCLE_CREATED	Facturation ponctuelle (commande) ou mensuelle	Domaine
	CUSTOMERBILL_CREATED	Traitement du cycle de facturation	Domaine
	PAYMENT_PROCESSED	Réception du paiement de l'abonné	Domaine
	PAYMENT_DELINQUANT	Client délinquant à gérer	Compensation
Ligne	SERVICE_VERIFIED	Vérification de la demande de portabilité ou d'activation	Domaine
	SERVICE_PROVISION_OK	Activation d'une ligne mobile avec ou sans portabilité	Domaine
	SERVICE_PROVISION_KO	L'activation d'une ligne mobile échoue	Compensation

Carte des flux (— trait plein : REST synchrone ; • — gras : appel synchrone vers un système fourni via adapter ; ... pointillés : événement asynchrone via le broker).



Catalogue de référence des événements de la saga (détail des schémas en Annexe A).

LOG430 — Phase 2 (event-driven) · Été 2026
© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

OfferingsVerified	Catalogue	Chorégraphe	Domaine
OfferingsInvalid	Catalogue	Chorégraphe	Compensation
StockDecreased	Catalogue	Chorégraphe	Domaine
StockIncreased	Catalogue	Chorégraphe	Compensation
OrderSagaSubmitted	Chorégraphe	Audit, Commande	Domaine
OrderValidationsOk	Chorégraphe	Commande	Domaine
OrderValidationsKo	Chorégraphe	Audit, Client	Compensation
DoProvisionActions	Chorégraphe	Audit, Ligne	Domaine
DoInvoicing	Chorégraphe	Audit, Facturation	Domaine
OrderSagaEnded	Chorégraphe	Audit, Client	Domaine
BuyerVerified	Client	Chorégraphe	Domaine
BuyerInvalid	Client	Chorégraphe	Compensation
PartyIdentified	Client	Audit	Domaine
OrderDraft	Commande	Client, Catalogue, Ligne	Domaine
OrderCreated	Commande	Audit, Catalogue, Chorégraphe	Domaine
OrderCanceled	Commande	Audit, Catalogue, Chorégraphe, Client	Compensation
BillcycleCreated	Facturation	Audit	Domaine
CustomerbillCreated	Facturation	Audit, Client	Domaine
PaymentProcessed	Facturation	Audit, Client, <i>Commande</i> , <i>Chorégraphe</i>	Domaine
PaymentDelinquant	Facturation	Audit, Client, Ligne, Commande	Compensation
ServiceVerified	Ligne	Chorégraphe	Domaine
ServiceProvisionOk	Ligne	Chorégraphe	Domaine
ServiceProvisionKo	Ligne	Audit, Client, <i>Chorégraphe</i>	Compensation

Note : les consommateurs *en italique* doivent implémenter les événements qui doivent aussi être ajoutés au AsyncAPI (il en manque 3).

4. Stratégie de solution — passage à l'événementiel

Objectif Arc42 — Résumer les décisions fondamentales de la transition : style événementiel, choix du broker, outbox pattern, saga chorégraphiée, cohabitation avec le REST synchrone.

4.1 Style architectural événementiel (chorégraphie)

La Phase 2 retient une saga chorégraphiée : chaque service réagit aux événements et en émet de nouveaux, sans coordinateur central. Justifiez ce choix face à l'orchestration.

Critère	Chorégraphie (retenue)	Orchestration (écartée)
Couplage	Dans une saga choréographiée, le couplage est faible entre les services car il n'y pas d'entité centrale qui gère la communication entre les services. Chaque service est responsable d'émettre des événements à un bus d'événement et de réagir en conséquence aux événements des autres services. Cela permet au système d'opérer en mode asynchrone, certains services peuvent se permettre d'être en panne sans pourtant faire échouer le système en entier. Un événement émis reste dans le bus d'événement pour être consommé.	La saga orchestration sert de point central de communication entre les services lors d'une transaction. Il est couplé aux apis REST des services avec lesquels il communique. C'est lui qui orchestre chaque étapes à suivre du début à la fin dépendemment des états dans lequel il se retrouve selon le retour d'un service. Ce type d'architecture en est mode synchrone, il dépend sur la disponibilité entière du système afin de pouvoir opérer, sinon il échoue car il ne reçoit aucun état des services.
Complexité du flux	La logique est répartie dans chaque service. Chacun est configuré pour réagir aux événements auxquels il devrait être souscrit pour effectuer les opérations nécessaires pour compléter une transaction. Les flux transactionnel est beaucoup plus complexe et nécessite des schémas de données pour bien fonctionner, sans lesquels les services pourraient	La logique d'une transaction est centralisé chez l'orchestrateur, il est responsable de tout gérer entre les services. Le flux est géré par une machine à état finie qui s'exécute à un seul endroit. Il est facile à mettre en oeuvre, à contrôler, déboguer et observer. Les appels sont succints et prévisibles et se font par REST HTTP.
Observabilité	L'observabilité pour un système en style événementiel s'avère plus compliqué qu'un système orchestrateur comme les informations sont dispersées sur plusieurs services (manque de système central). Le système de tracabilité (Jaeger) doit être configuré pour corréler les événements avec un identifiant de trace afin d'observer le déroulement et reconstruire la transaction avec le journal événementiel de kafka.	En orchestration, le flux d'événements est contrôlé et maintenu par l'orchestrateur, alors il est plus facile de garder des traces de l'état du système et de trouver l'endroit exact du point d'échec si la journalisation a correctement été configurée. L'observabilité du système se fait par analyse des journaux, des métriques (Prometheus/Grafana) et des traces synchrones (Jaeger). Le déroulement s'observe comme si on suit un fil d'ariane du début de

	L'observabilité des systèmes se fait par métriques de la même façon qu'avec le système orchestré. Pour comprendre ce qui a causé un point d'échec sur un service, la corrélation entre les traces Jaeger, les journaux système et les métriques permettent reconstruire l'état de la situation problématique. Il existe des cas de «silent failure» où un événement n'a jamais été diffusé ou jamais été consommé ce qui rend le troubleshooting plus comparé comme il n'y a même pas de message d'erreur au moins. <i>À placer dans les risques</i>	la transaction jusqu'à la fin.
Justification du choix	On souhaite convertir la BSS dans un style architectural en saga choréographie car le système a besoin d'être opérable 24 / 7 et de soutenir une charge élevée. La transition vers un style architectural en choréographie nous permet de mettre à l'échelle plus facilement et de répondre plus rapidement (presque en temps réel) aux usager de par sa nature asynchrone. De plus, la mise à l'échelle est facilitée par l'ajout d'instances, sans nécessiter de modification au répartiteur de charge, car les instances s'inscrivent au bus de messagerie et n'ont pas besoin d'être connectées à un répartiteur de charge. L'architecture distribue de façon atomique les opérations à effectuer pour le processus de la saga et optimise le temps de traitement et de blocage de chaque service. C'est l'option retenue pour assurer que la BSS soit rapide, fortement disponible et sécuritaire.	L'architecture orchestrée est de nature synchrone et naturellement plus lente car toutes la chaine des opérations à faire pour remplis la saga doivent s'exécuter avant de libérer les ressources. La mise à l'échelle se fait en configurant des répartiteurs de charge dans le système qui deviennent eux-même des points de défaillance. L'interface avec le système (la ressource d'accès) bloque à partir du moment où le message est reçu jusqu'à ce que le traitement soit terminé, l'équivalent de plusieurs secondes. Les résultats des tests de la saga orchestrée démontrent que le système n'arrive pas à bien répondre sous tension.

4.2 Broker de messages

Décision	Choix retenu	Justification (→ ADR)
Broker	Kafka	→ ADR-007
Topologie	1 grappe Kafka avec 3 noeuds à l'intérieur. Il y a 1 topic kafka par état de la chorégraphie SAGA, ainsi, on retrouve 25 topics événementiels dans l'application. Il y a 4 partitions par topics. Les schémas de messages sont documentés selon le format AsyncAPI dans un fichier <i>asynccapi.yml</i>. La clé de message Kafka utilisée pour un message est l'identifiant de corrélation de la transaction. C'est la même clé qui est utilisée pour tracer les transactions dans le système.	Le choix d'avoir un canal (<i>topic</i>) événementiel par état de la chorégraphie, pour un traitement granulaire et isolé des changements d'état, vient permettre un gain de performance au niveau des partitions de chaque <i>topic</i>. Puisque les <i>topics</i> sont granulaires, le nombre de partitions par <i>topic</i> peut être réduit. Ainsi, on choisit d'implémenter 4 partitions par <i>topic</i>, estimant que ça puisse répondre au besoin des consommateurs. L'utilisation de l'identifiant de corrélation transactionnelle comme clé simplifie la lecture en permettant de lire tous les messages d'une même transaction, pour un même <i>topic</i>, dans l'ordre, dans une même partition. Ceci augmente la performance.
Rétention / rejeu	La rétention des messages d'un topic est configurée à 2 Go. Le rejeu se fait par consommation des événements passés depuis le début et leur ré-exécution dans le système chorégraphié.	On estime que la rétention de 2Go de données de message par topic, dans le journal événementiel, est suffisant pour permettre un audit à long terme des messages et de l'état de l'application.
Sérialisation	La sérialisation des messages est faite en JSON et le registre des schémas est standardisé via le standard industriel AsyncAPI et son fichier <i>asynccapi.yml</i>. Les schémas sont renforcés lors de l'écriture pour garantir le respect du contrat AsyncAPI par chaque service.	L'utilisation d'un standard garantit une meilleure interopérabilité et un effort de maintenance moindre. Ainsi on a décidé d'utiliser AsyncAPI et le format JSON pour la définition et la sérialisation des messages.

4.3 Outbox pattern

Le outbox pattern garantit qu'un événement métier est à la fois réalisé et publié dans une même transaction atomique. Ainsi, si et seulement si la transaction métier est validée : l'événement est

écrit dans une table « outbox » dans la même transaction que la donnée, puis un polling périodique le publie vers le broker Kafka et marque l'événement comme publié dans la base de données.

L'idempotence de la publication est garantie par une clé de corrélation ajoutée au message. Bien qu'il se peut que la publication soit faite plus d'une fois – *at-least-once* (si le marquage comme *publié* échoue p.ex), le consommateur de message n'agira qu'une seule fois grâce à la clé d'idempotence. Ainsi, le patron Outbox a une caractéristique *exactly-once*.

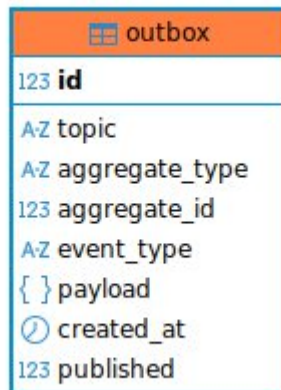


Figure 2 Schéma de la table Outbox

4.4 Saga chorégraphiée & compensation

L'architecture garantit une cohérence éventuelle de la saga chorégraphiée soit par achèvement de la saga ou par compensation. La production et la consommation de messages s'enchaînent dans le système en utilisant les patron outbox et d'idempotence pour garantir un *exactly-once* qui permet à une transaction d'aboutir même en cas de panne de parties du système.

Lorsque des exceptions se produisent dans le système, des événements de compensation sont émis pour défaire l'état du système altéré jusque-là par la transaction. Alors, le flux événementiel permet aux différents services de faire des actions compensatoires et qu'au terme du traitement éventuel garanti de la saga l'état du système ait été remis à ce qu'il était avant l'exécution de la saga.

La saga cible de notre système est la commande. Les étapes nominales sont illustrées à la Figure 1 et énumérées ci-dessous. Il y a 7 étapes :

- Création d'une commande brouillon
- Valider la commande soumise
- Diminuer les stocks
- Activer un service
- Créer la facture
- Recevoir le paiement
- Compléter la commande

Les événements de succès sont :

Événement de succès	Signification
AlerteFraude	Une fraude est détectée par le système anti-fraude

AbusUsage	Un abus est détecté par le système anti-fraude
OfferingsVerified	Les offres d'une commande sont vérifiées dans le catalogue
StockDecreased	Le stock de la commande a été déduit de l'inventaire
OrderSagaSubmitted	Une nouvelle commande client est arrivée
OrderValidationsOk	Toutes les validations de la commandes sont positives
DoProvisionActions	Lancement de l'activation de la ligne
DoInvoicing	Lancement de la facturation
OrderSagaEnded	La commande est complétée
BuyerVerified	L'acheteur est vérifié
PartyIdentified	L'individu qui passe la commande est identifié
OrderDraft	Le brouillon de commande est disponible
OrderCreated	La commande valide est crée
BillcycleCreated	La commande peut être facturée
CustomerbillCreated	La commande est facturée
PaymentProcessed	Un paiement est reçu pour la commande
ServiceVerified	La disponibilité de la ligne dans la commande est vérifiée
ServiceProvisionOk	La ligne commandée est activée

De même, les événements d'échec sont :

Événement de succès	Signification
OfferingsInvalid	Il y a des offres invalides dans la commande reçue
StockIncreased	Les stocks ont été augmentés suite à une erreur
OrderValidationsKo	Au moins une validation de commande est négative
BuyerInvalid	L'acheteur est invalide
OrderCanceled	La commande est annulée
PaymentDelinquant	Le paiement n'a pas été reçu depuis longtemps
ServiceProvisionKo	L'activation de ligne a échoué

Enfin, l'état final attendu de la chorégraphie est ORDER_SAGA_COMPLETED et celui de la commande : ORDER_COMPLETED ou ORDER_CANCELED.

4.5 Cohabitation REST synchrone ↔ événementiel

Précisez ce qui reste synchrone (requêtes utilisateur via API Gateway) et ce qui devient asynchrone (propagation inter-services par événements).

Les requêtes REST des utilisateurs retournent désormais un code HTTP 202 – Accepted lorsqu'accepté pour traitement par le système. La cohabitation REST et événementiel est en symbiose alors que l'API v1 de l'orchestration (synchrone) est maintenu opérationnel tandis que l'API v2 de l'orchestration (asynchrone) est mise en opération.

La notification de l'état final est reçu par notification courriel.

Tous les retours d'événements REST sont disponible dans la documentation de la phase I via la documentation OpenAPI.

4.6 Intégration des systèmes fournis (HLR/HSS & PortabilityHub) — obligatoire

Deux systèmes externes sont fournis par l'encadrement et leur utilisation est obligatoire : le HLR/HSS (free5GC) pour le provisionnement des abonnés et le PortabilityHub pour la gestion de la portabilité des numéros. Conformément à l'architecture hexagonale, ces systèmes ne sont jamais utilisés directement par le domaine. La couche application dépend uniquement de ports (SubscriberProvisioningPort), tandis que les détails des protocoles, des contrats et des formats d'échange sont encapsulés dans des adaptors jouant le rôle de couche anticorruption (ACL).

Système fourni	Utilisé par (contexte)	Rôle dans la saga	Intégration & fiabilité
HLR / HSS	Lignes & Services	Provisionner le MSISDN lors de l'activation (UC-03)	Adapter synchrone, appel idempotent, timeout + retry, échec → ActivationÉchouée (compensation)
PortabilityHub	Lignes & Services (Commandes)	Port-in / port-out du numéro (UC-03)	Adapter + idempotencyKey ; statut final asynchrone (PORTED / REJECTED / EXPIRED) via callback ; échec → PortabilitéÉchouée (compensation)

Le provisionnement est réalisé par l'adaptor Free5GCWebUIAdapter, qui implémente le port SubscriberProvisioningPort. Le domaine manipule uniquement les objets métier (MobileLine, Supi, Msisdn) et ne dépend jamais du format JSON propre à free5GC. Le profil d'abonné attendu par la WebUI est construit par build_subscriber_profile(), ce qui isole le domaine du modèle du fournisseur. Les interactions avec la WebUI free5GC sont les suivantes :

Endpoint	Description
POST /api/login	Authentification et obtention du jeton JWT.
GET /api/subscriber/{supi}/{plmn}	Vérification de l'existence de l'abonné (idempotence).
POST /api/subscriber/{supi}/{plmn}	Création du profil d'abonné dans le HLR/HSS.
DELETE /api/subscriber/{supi}/{plmn}	Suppression du profil lors d'une compensation.

L'adapteur conserve le jeton JWT en mémoire. Lorsqu'une requête retourne 401 Unauthorized, un nouveau jeton est obtenu automatiquement et la requête est rejouée une seule fois. Toutes les requêtes utilisent un timeout configurable (WEBUI_TIMEOUT_SECONDS, 10 secondes par défaut). Les exceptions réseau et les erreurs HTTP sont converties en ProvisioningError, ce qui provoque l'échec de l'étape de saga et la compensation des abonnés créés pendant l'opération.

Le PortabilityHub expose un contrat événementiel conforme à TMF Event Management v4, décrit par un document AsyncAPI. Chaque message est une enveloppe EventNotification contenant notamment eventId, eventType, correlationId, eventTime, source ainsi que le payload métier event. Le flux de portabilité repose sur les topics Kafka suivants :

- portability.port-in-requested
- portability.out-validation-needed
- portability.out-validated
- portability.order-approved
- portability.order-rejected
- portability.port-completed
- portability.order-cancelled

Les événements sont corrélés à l'aide du champ correlationId. Lors de la publication de la demande initiale (PortInRequestedEvent), cette valeur correspond au premier eventId. Après la création de l'ordre par le Hub, elle devient l'identifiant de l'ordre de portabilité et demeure inchangée pendant tout le traitement, ce qui permet de suivre l'ensemble du parcours métier jusqu'à son état final (approved, rejected, port_completed ou cancelled).

Cette même clé est utilisée pour la traçabilité.

5. Vue des blocs de construction

Objectif Arc42 — Décomposition statique enrichie des composants événementiels : broker, producteurs, consommateurs, relais outbox, participants de la saga. Couvre les vues Logique et Développement de 4+1.

5.1 Vue d'ensemble : services + broker

Similairement à la phase I, le client communique directement avec chaque service à partir de l'API Gateway. Dans un style architectural événementiel, le mode de communication devient asynchrone à travers le bus d'événements Kafka.

Origine	Destination	Type	Description
Commandes	Topic Commandes	Kafka (Publie)	Émet des événements liés au cycle de vie d'une commande (créée, brouillon, annulée).
Clients	Topic Clients	Kafka (Publie)	Émet des événements liés à l'identité (acheteur vérifié, abonné identifié).
Lignes	Topic Lignes	Kafka (Publie)	Émet des événements d'activation de ligne, de vérification de service ou de tarification.
Facturation	Topic Facturation	Kafka (Publie)	Émet des événements de production de facture, cycle de facturation ou paiements.
Audit	Topic Audit	Kafka (Publie)	Émet des événements critiques de sécurité (alertes de fraude, abus d'usage).
Catalogue	Topic Catalogue	Kafka (Publie)	Émet des événements liés aux offres (vérifiées, invalides) ou à la gestion des stocks.
Chorégraphie	Topic Choregraphie	Kafka (Publie)	L'orchestrateur émet des <i>commandes d'action</i> (ex: fait-le provisioning, fait la facturation) et des statuts globaux de saga.

Topic Commandes	Clients, Lignes, Audit, Catalogue, Chorégraphie	Kafka (Consomme)	L'orchestrateur (Chorégraphie) écoute pour démarrer la saga. Les autres pour préparer leurs données ou auditer.
Topic Clients	Audit, Chorégraphie	Kafka (Consomme)	L'orchestrateur écoute pour savoir si l'acheteur est valide. L'Audit écoute pour des raisons de sécurité.
Topic Lignes	Audit, Clients, Chorégraphie	Kafka (Consomme)	L'orchestrateur écoute pour savoir si le provisioning a réussi ou échoué.
Topic Facturation	Audit, Clients, Commandes, Lignes, Chorégraphie	Kafka (Consomme)	L'orchestrateur écoute pour confirmer le paiement. Les Lignes écoutent pour suspendre le service en cas de non-paiement.
Topic Audit	Clients, Facturation, Lignes, Chorégraphie	Kafka (Consomme)	Si l'audit détecte une fraude, tous ces services écoutent pour potentiellement bloquer les transactions ou la ligne.
Topic Catalogue	Chorégraphie	Kafka (Consomme)	L'orchestrateur attend la confirmation de la vérification de l'offre et du stock avant de passer à l'étape suivante.
Topic Choregraphie	Audit, Clients, Commandes, Facturation, Lignes	Kafka (Consomme)	C'est ici que l'orchestrateur distribue ses "ordres" : Facturation écoute l'ordre de facturer, Lignes écoute l'ordre d'activer, etc

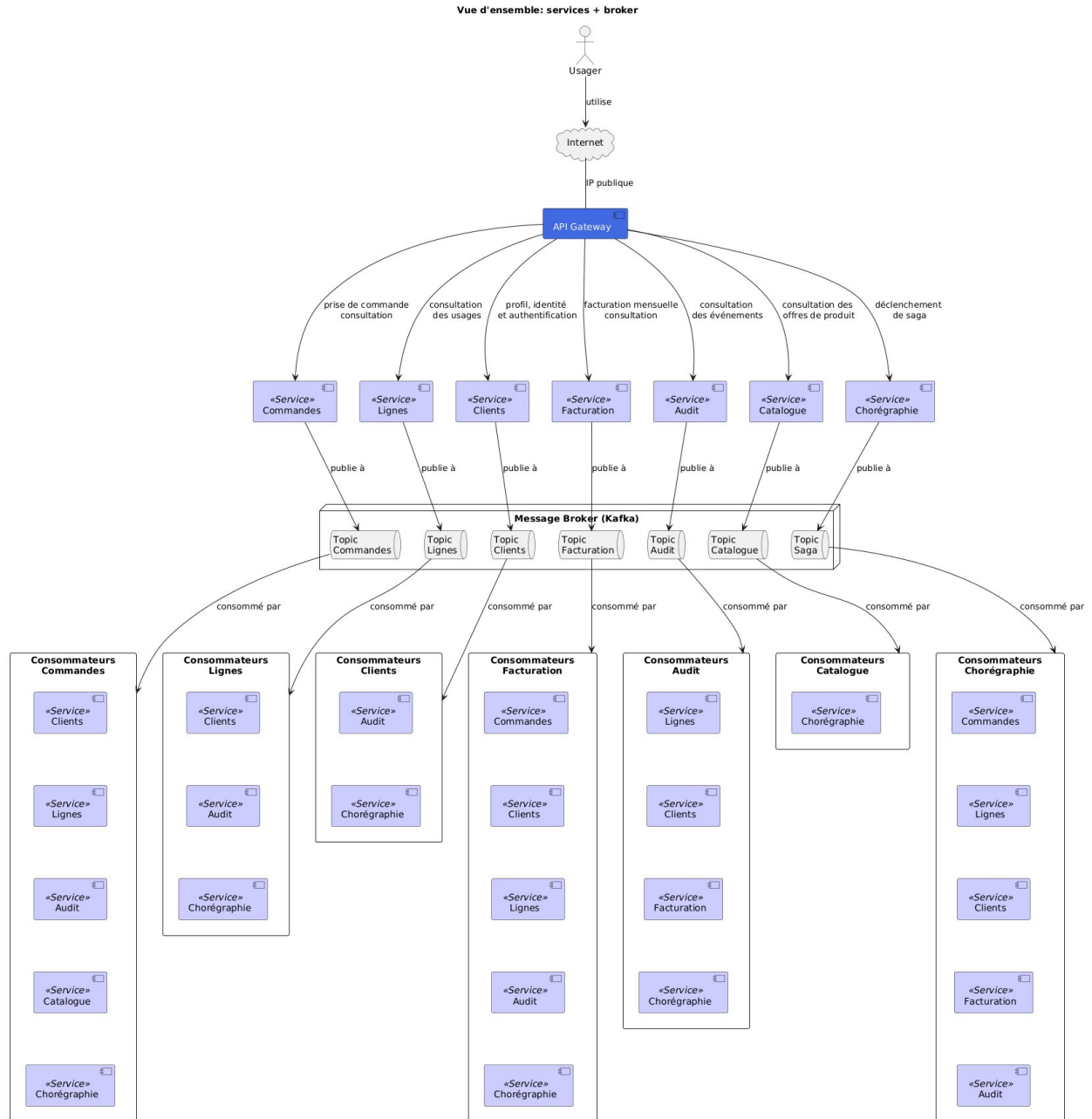


Figure 3 - Vue d'ensemble: service + broker

5.2 Composants événementiels d'un service

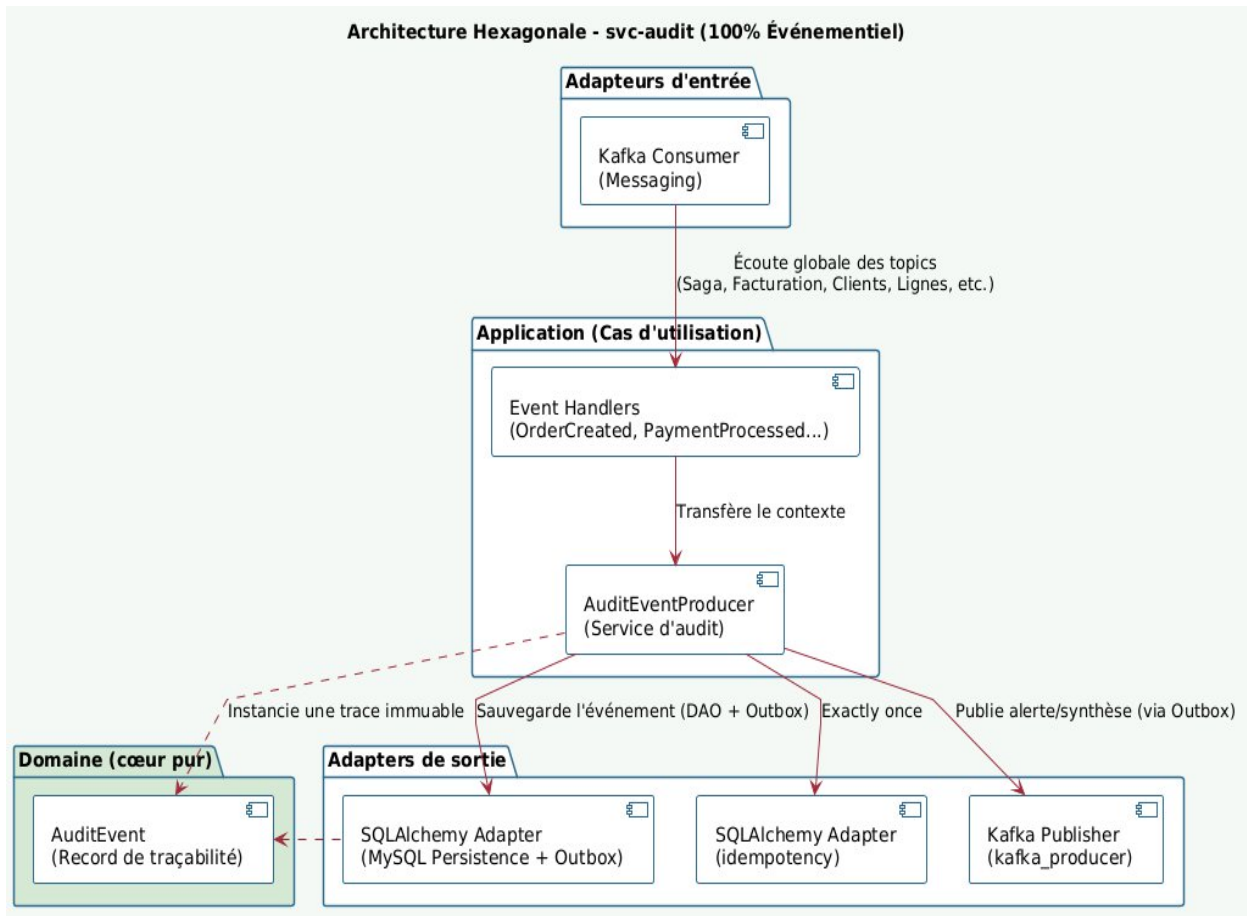


Figure 4 - Schéma d'architecture hexagonale pour svc-audit

Cas particulier du contexte Lignes & Services : outre les ports événementiels, il expose deux adaptateurs sortants obligatoires vers les systèmes fournis — un adaptateur HLR/HSS (provisionnement MSISDN) et un adaptateur PortabilityHub (port-in/port-out) — chacun protégé par une couche anticorruption (ACL). Voir §4.6.

5.3 Modèle & versionnement des schémas d'événements

Événement	Champs clés (payload)	Version / compatibilité
DetectionFraude	eventType, eventTime, description, priority, timeOccurred, partyRef, MSISDN, correlationId, domain, event, party	v1
AbusUsage	usageType, eventTime, subscriberRef, MSISDN, timeOccurred, correlationId	v1
OfferingsVerified	productOfferings, orderId	v1
OfferingsInvalid	invalidproductofferings, orderId	v1

StockDecreased	affectedProductOfferings, orderId	v1
StockIncreased	affectedProductOfferings, orderId	v1
OrderSagaSubmitted	timeSubmitted, order, userId	v1
OrderValidationsOk	orderId, validation	v1
OrderValidationsKo	orderId, validation	v1
DoProvisionActions	orderId, MSISDN	v1
DoInvoicing	orderId, billCycleType	v1
OrderSagaEnded	status, activation, customerbillId, orderId, paymentLink	v1
BuyerVerified	partyRef, orderId, validation	v1
BuyerInvalid	partyRef, orderId, validation	v1
PartyIdentified	partyRef, identificationTime, validation	v1
OrderDraft	orderState, timeOccured, order	v1
OrderCreated	orderState, timeOccured, order	v1
OrderCancelled	orderState, timeOccured, order	v1
BillcycleCreated	billcycleId	v1
CustomerbillCreated	customerbillId, totalAmount	v1
PaymentProcessed	ustomerbillId, paidAmount, amountDue	v1
PaymentDelinquant	userId, customerbillId, amountOverdue	v1
ServiceVerified	orderId, serviceRef	v1
ServiceProvisionOk	service, orderId, validation	v1
ServiceProvisionKo	service, orderId, validation	v1

5.4 Enveloppe commune des messages

Les messages ont tous une structure commune qui est composée de :

- aggregate_type
- aggregate_id
- eventId
- eventType,
- traceId
- occurredAt

6. Vue d'exécution — flux événementiels

Objectif Arc42 — Comportement dynamique des sagas : scénario nominal, compensation, et fiabilité outbox → publication → consommation. Couvre la vue Processus (C&C) de 4+1.

6.1 Saga chorégraphiée — scénario nominal

Interaction de bout en bout pour la prise de commande d'un forfait. La requête utilisateur reçoit une réponse immédiate (202 « en cours ») ; les effets sont ensuite propagés de façon asynchrone par événements. Chaque service consomme l'événement qui le concerne, exécute son étape locale et publie l'événement suivant — sans orchestrateur central.

Dans le scénario nominal, les messages transitent sur le bus de messagerie en utilisant 15 topics différents qui sont, dans l'ordre d'apparition, attendus :

#	Nom	Topic AsyncAPI + Kafka
1	orderSagaSubmitted	saga.order-submitted
2	orderDraft	commande.order-draft
3	offeringsVerified	catalogue.offerings-verified
4	serviceVerified	lignes.service-verified
5	buyerVerified	client.buyer-verified
6	orderValidationsOk	saga.order-validations-ok
7	orderCreated	commande.order-created
8	stockDecreased	catalogue.stock-decreased
9	doProvisionActions	saga.order-do-provision
10	serviceProvisionOk	lignes.service-provision-ok
11	doInvoicing	saga.order-do-invoicing
12	billcycleCreated	facturation.billcycle-created
13	customerBillCreated	facturation.customerbill-created
14	paymentProcessed	facturation.payment-processed
15	orderSagaEnded	saga.order-ended

Les en-têtes propagées dans la chorégraphie sont :

aggregate_type : le type de service ayant publié le message

aggregate_id : l'identifiant de l'instance de service ayant publié le message

correlation_id : clé d'idempotence de la transaction, et traceId.

La clé d'idempotence est fournie par l'Abonné lors de la soumission de la commande. Et est incluse dans le lien de paiement lorsque l'abonnée l'effectue dans un second temps.

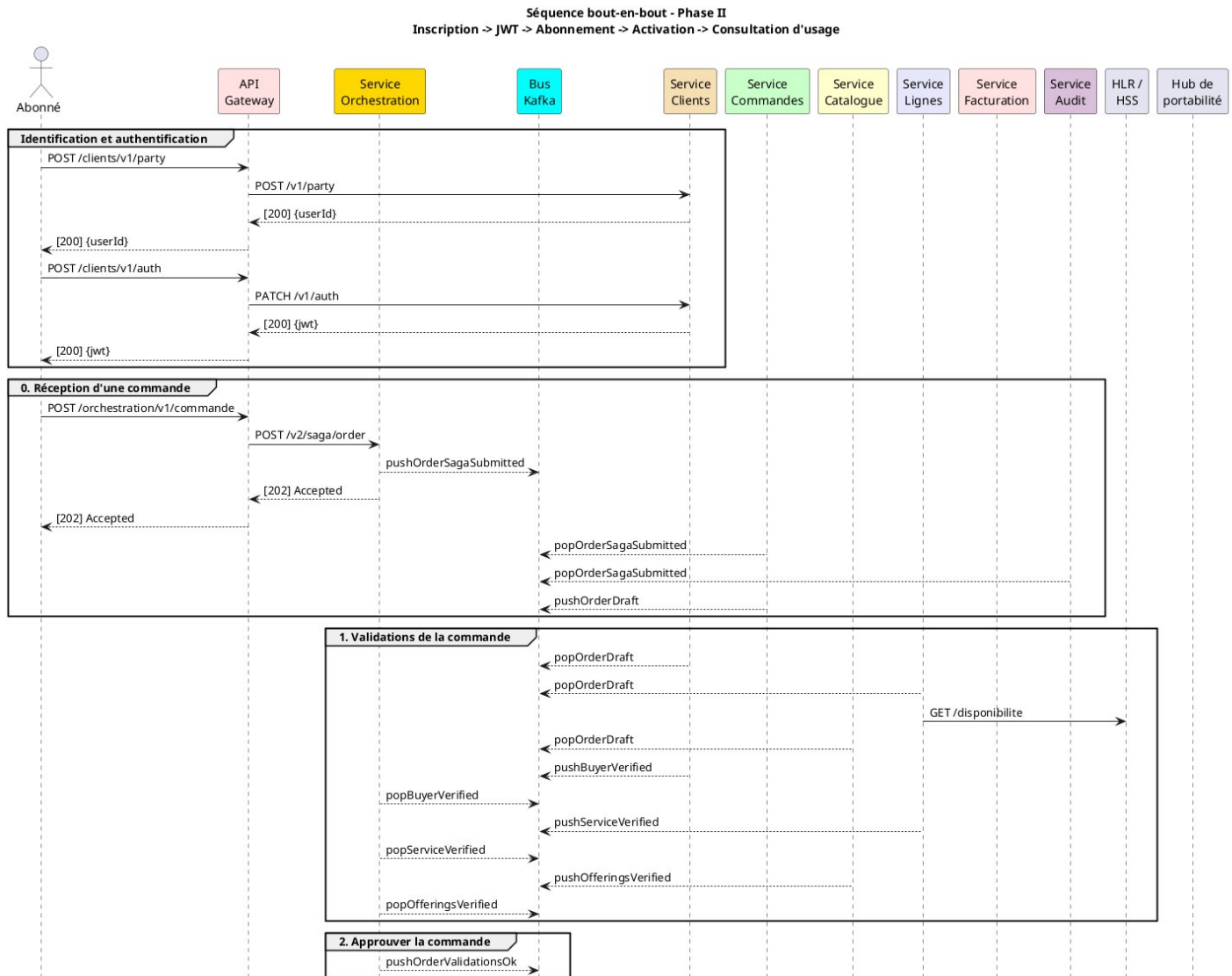


Figure 11 Séquence chorégraphiée du scénario nominal - 1 de 2

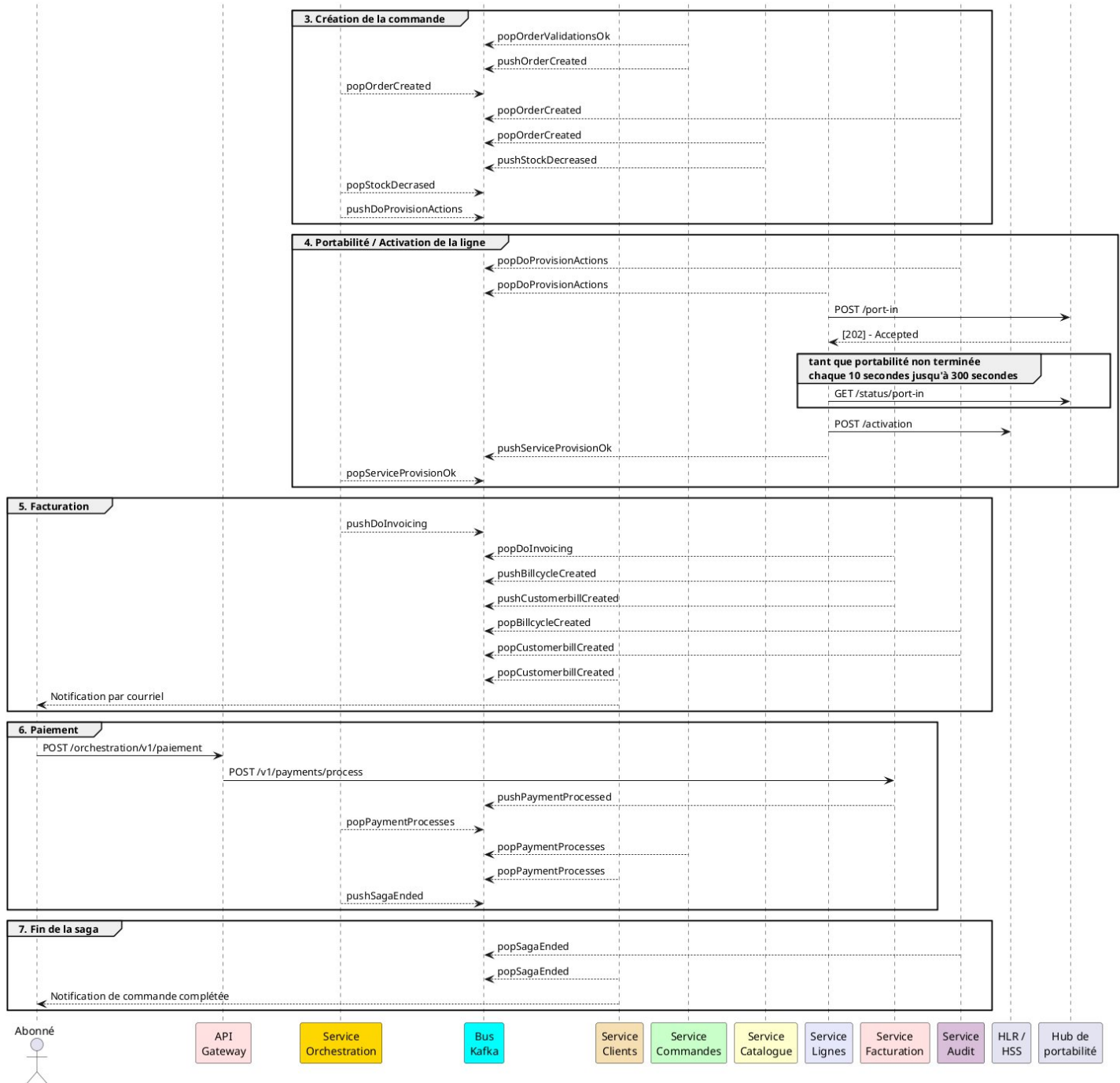


Figure 12 Séquence chorégraphiée du scénario nominal - 2 de 2

Le contrat d'activation de l'adaptateur HLR suit la structure suivante :

- plmnID : Identifiant du réseau mobile (PLMN).
- ueld : SUPI/IMSI de l'abonné.
- AuthenticationSubscription : Paramètres d'authentification 5G-AKA (Ki, OPc, SQN, AMF).
- AccessAndMobilitySubscriptionData : MSISDN ('gpsis'), AMBR et S-NSSAI autorisés.
- SessionManagementSubscriptionData : Configuration des sessions PDU (DNN 'internet', QoS, débit, 5QI).
- SmfSelectionSubscriptionData : Sélection du SMF et DNN disponibles.
- AmPolicyData : Politiques d'accès et de mobilité.

- SmPolicyData : Politiques de gestion des sessions.
- FlowRules : Règles de flux (vide par défaut).
- QosFlows : Flux QoS (vide par défaut).
- ChargingDatas : Données de taxation (vide par défaut).

```
{
  "plmnID": "20893",
  "ueld": "imsi-208930000000001",
  "AuthenticationSubscription": { ... },
  "AccessAndMobilitySubscriptionData": { ... },
  "SessionManagementSubscriptionData": [ ... ],
  "SmfSelectionSubscriptionData": { ... },
  "AmPolicyData": { ... },
  "SmPolicyData": { ... },
  "FlowRules": [],
  "QosFlows": [],
  "ChargingDatas": []
}
```

[À compléter — adaptez les services, topics et handlers à votre implémentation ; précisez les topics/clés de partition et les en-têtes propagés (traceld), ainsi que le contrat de l'adapter HLR.]

6.2 Compensation (échec d'une étape)

Lorsqu'une étape échoue (ici, paiement refusé), la saga ne « rollback » pas globalement : chaque service défait localement son effet via un événement de compensation, jusqu'à un état terminal cohérent (commande annulée, réservation de ligne libérée). La facturation arrive en toute fin de processus, alors le refus de paiement oblige à compenser toute la transaction.

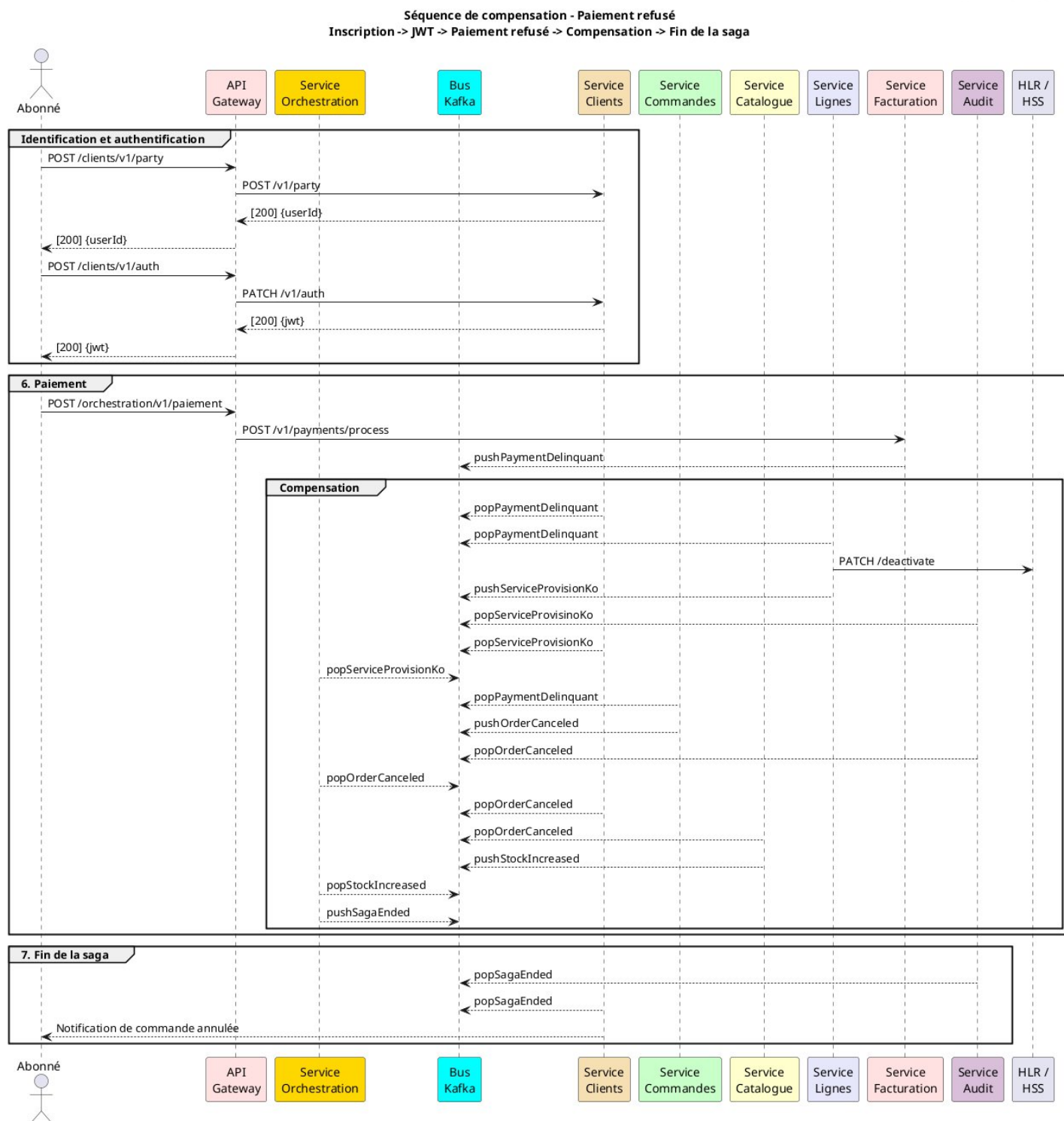


Figure 13 Séquence de compensation - Paiement refusé

La clé d'idempotence de la transaction de la commande est fournie par l'Abonné via son lien de paiement. C'est la même clé qui a servi pour démarrer la saga chorégraphiée alors elle est utilisée pour compléter le chemin et permettant la traçabilité.

Les Sagas orphelines sont gérées par un processus qui vérifie périodiquement les sagas de chorégraphie qui sont actives et non terminées. Lorsqu'un délai d'exécution supérieur à 20 minutes est détecté, un disjoncteur applicatif fait compenser la saga automatiquement.

6.3 Fiabilité outbox → publication → consommation

Le outbox pattern rend la publication fiable : l'événement est écrit dans la même transaction que la donnée métier, puis un relais le publie (livraison at-least-once). Le consommateur déduplique par clé d'idempotence pour un effet unique, même en cas de rejeu ou de doublon, obtenant ainsi le *exactly-once*.

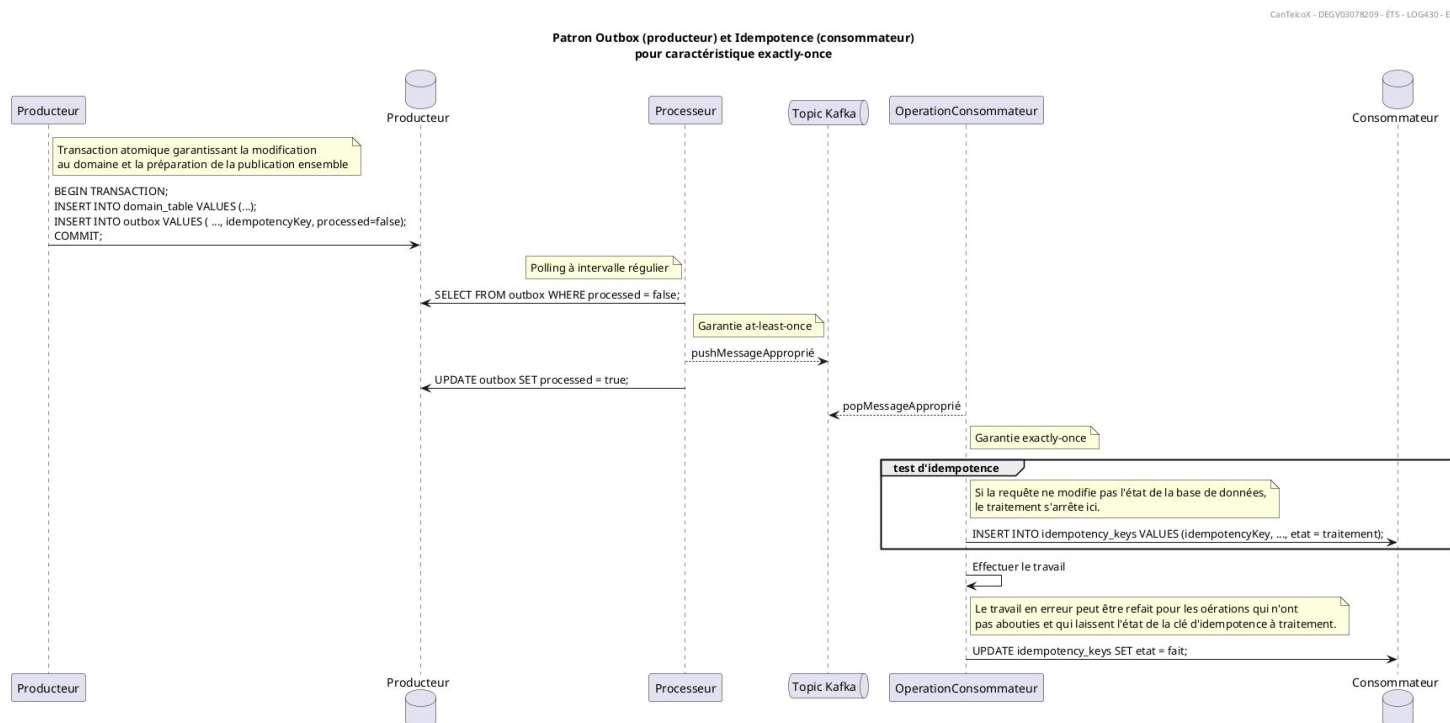


Figure 14 Séquence Outbox et Idempotence pour caractéristique exactly-once

La table de déduplication utilisée est une table `idempotency_key` telle qu'utilisée en phase I.

idempotency_keys	
AZ	idempotency_key
AZ	request_hash
AZ	status
123	response_code
AZ	response_body
🕒	created_at
🕒	updated_at

Figure 15 Schéma de la table `idempotency_keys`

Persistance des données dans l'Outbox

Les données sont conservées dans la table Outbox. Quand elles sont publiées, elles sont immédiatement retirées de celle-ci. Les données n'ont pas de date de péremption: tant qu'elles ne sont pas publiées, elles restent dans la table.

6.4 Portabilité via le PortabilityHub fourni (port-in)

Le port-in d'un numéro s'appuie obligatoirement sur le PortabilityHub fourni. Le contexte Lignes & Services initie le port-in via un adapter (avec idempotencyKey) ; le hub répond d'abord par un ACK, puis, après la fenêtre de portabilité, notifie le statut final de façon asynchrone (callback). Selon le résultat, la saga publie NuméroPorté (succès) ou PortabilitéÉchouée (compensation → annulation de la commande).

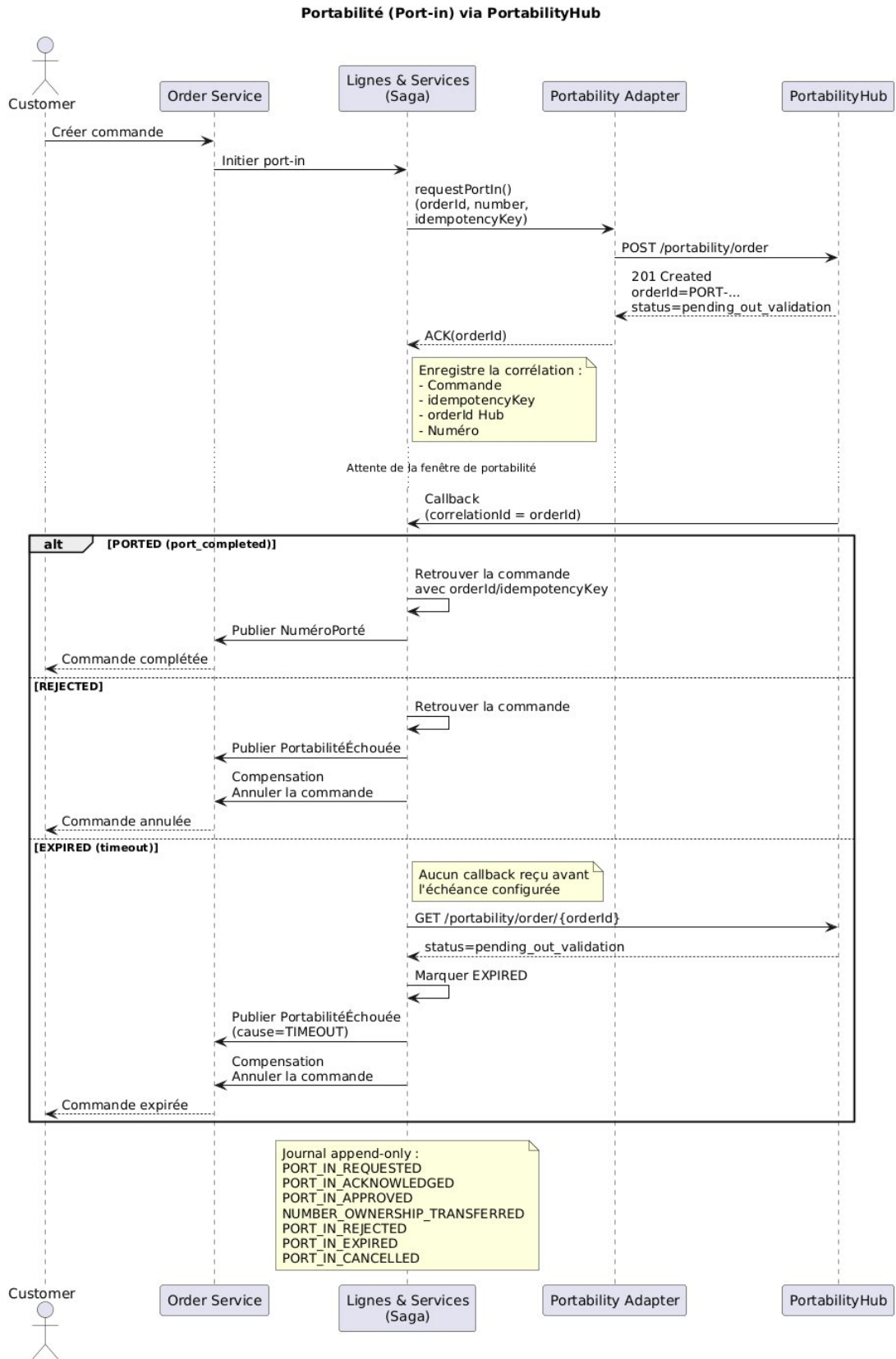


Figure 16 Portabilité via le portability hub

Le processus de portabilité d'un numéro est intégré à la saga chorégraphiée afin de permettre le transfert d'un numéro existant depuis un autre opérateur sans bloquer le reste du traitement. Lorsqu'une commande nécessite un port-in, le contexte Lignes & Services invoque le PortabilityHub au moyen d'un adaptateur dédié. La requête contient une idempotencyKey ainsi que le orderId, permettant d'éviter la création de demandes en double et d'assurer la corrélation entre la commande métier et les événements retournés.

Le PortabilityHub répond immédiatement par un ACK, confirmant uniquement la réception de la demande. La décision finale est transmise de manière asynchrone par un callback lorsque la fenêtre de portabilité est terminée. Ce callback contient le statut final (PORTED, REJECTED ou EXPIRED) ainsi que les informations de corrélation permettant de retrouver la saga concernée.

À la réception du callback, le service Lignes & Services valide l'identité de la demande grâce au orderId et à la idempotencyKey, puis publie l'événement correspondant sur Kafka. En cas de succès (PORTED), l'événement ServiceProvisionOk (ou NuméroPorté) est publié afin de poursuivre normalement la saga. En cas d'échec (REJECTED ou EXPIRED), un événement ServiceProvisionKo (ou PortabilitéÉchouée) est émis afin de déclencher les mécanismes de compensation, notamment l'annulation de la commande, la restauration des ressources réservées et la notification du client.

Afin d'assurer la fiabilité du traitement, chaque callback est traité de façon idempotente. Si un même événement est reçu plusieurs fois, un seul effet métier est appliqué. Toutes les transitions d'état sont enregistrées dans un journal append-only, ce qui garantit une piste d'audit complète retraçant la demande de portabilité depuis sa création jusqu'à son état final. Cette approche facilite également le diagnostic des erreurs et la reconstruction d'une transaction lors de l'analyse des traces distribuées.

Cette intégration respecte les principes de l'architecture hexagonale : le PortabilityHub est considéré comme un système externe accessible uniquement via un adaptateur, ce qui isole le domaine métier des détails techniques de communication et facilite l'évolution ou le remplacement du fournisseur sans impact sur la logique applicative. Le projet implémente d'ailleurs le PortabilityHub comme un système distinct, avec sa propre API, ses schémas d'événements et son mécanisme d'outbox.

7. Vue de déploiement

7.1 Topologie de déploiement (delta)

Les répartiteurs de charge manquaient dans la phase I alors ils ont été ajoutés dans celle-ci à raison de 4 par service et 4 pour la passerelle API.

En plus, il y a trois conteneurs Kafka gérés avec KRaft. Les collecteurs de traces et Jaeger ont été intégrés en Phase I et on les documente quand même ici.

Composant ajouté	Rôle	Image / configuration
Broker de messages	Transport des événements	confluentinc/cp-kafka:7.5.0 1 grappe. 3 noeuds et 4 partitions par noeud.
Registre de schémas (opt.)	Contrôle de compatibilité des schémas	Les schémas sont contrôlés par des validations dans les services.
Collecteur de traces	Réception des spans OTel	jaegertracing/alli-in-one:latest Les traces sont récupérées par Jaeger via OTLP gRPC et HTTP
Backend de tracing	Visualisation des traces	jaegertracing/alli-in-one:latest
Répartiteurs de charge	Équilibrer la charge de la demande parmi les services disponibles	nginx:latest 4 répartiteurs devant api-gateway 4 répartiteurs devant service REST

La vue suivante illustre les frontières des composantes déployées. Tout en haut (cadre vert), l'api-gateway, les services de l'application et les répartiteurs de charge sur la couche dite applicative.

En-dessous, trois cadres pour la gestion d'identités et de données personnelles, le bus événementiel et les apis pour la portabilité et l'activation de ligne sur une couche de service.

Enfin, tout en bas, les ensembles de persistance et de cache ainsi que d'observabilité.

L'environnement de déploiement est Docker Compose et on souhaite migrer à Kubernetes dans l'avenir.

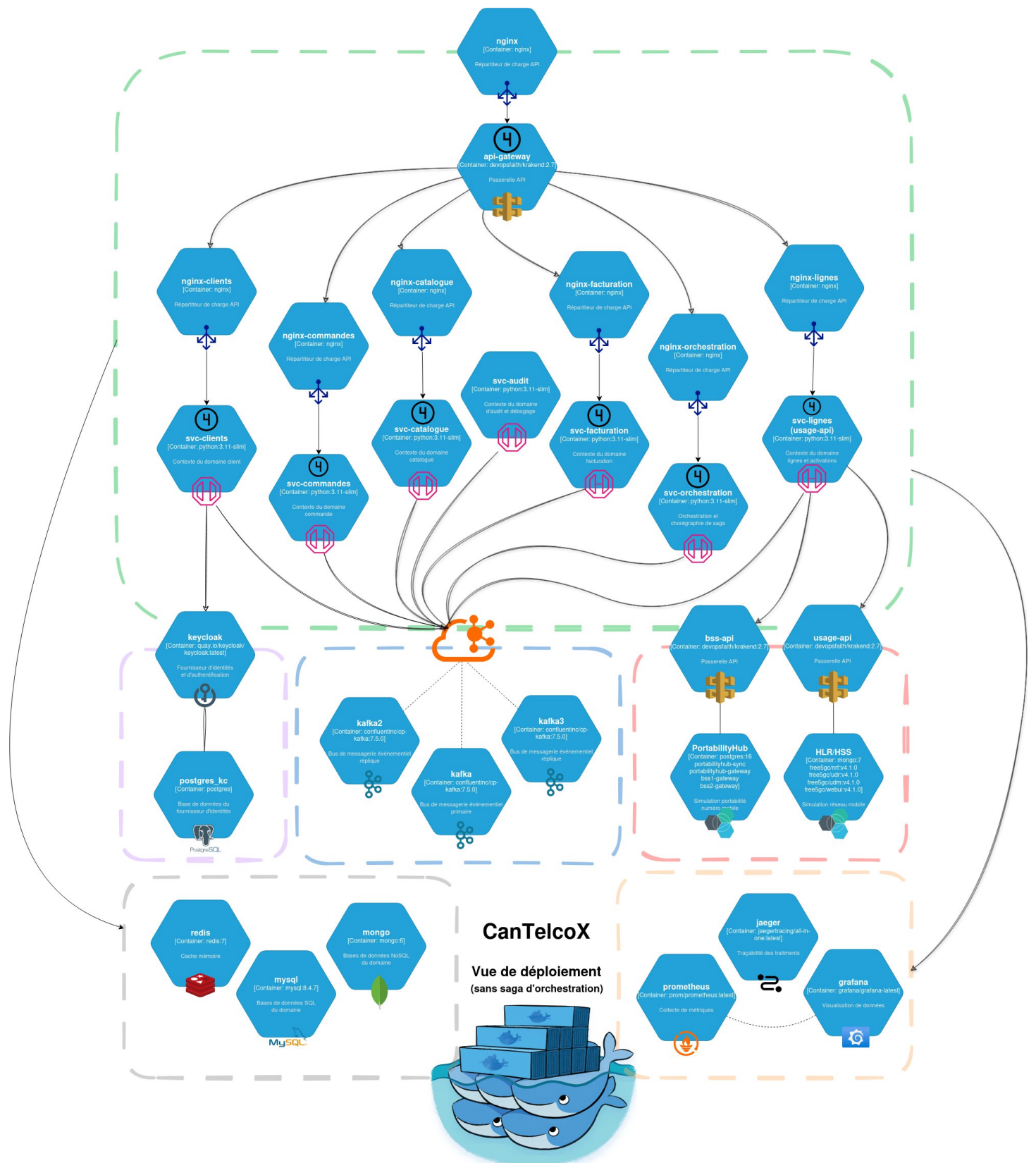


Figure 17 Vue de déploiement - diagramme enrichi - phase II

Réseaux

L'utilisation d'une passerelle API délimite les réseaux applicatifs. Ainsi, le réseau **cantelcox-network** est utilisé par la majorité des conteneurs. Les ensemble de conteneurs servant à de la simulation (HLR/HSS et PortabilityHub) ont leur propre sous-réseau et partagent un accès au réseau de l'application avec sa propre passerelle API.

Volumes

Chaque nouveau service a son volume propre et aucun nouveau volume supplémentaire n'est ajouté.

État de santé

La vérification de santé des services est faite via Docker. Voici un tableau qui donne les vérifications faites pour chaque service, leur fréquence, leur expiration et la dépendance entre les services. On omet ici les services inclus dans les simulateurs HLR/HSS et PortabilityHub.

Nom du conteneur	Dépendance	Règle de redémarrage	Test	Fréquence
mysql	-	unless-stopped	mysqladmin ping	Interval: 10s Timeout: 10s Retries: 20 Start: 1s
mongo	-	Unless-stopped	Mongosh ping	Interval: 5s Timeout: 5s Retries: 20 Start: 5s
redis	-	Unless-stopped	Redis-cli ping	Interval: 1m Timeout: 10s Retries: 20 Start: 10s
api-gateway	Svc-clients Svc-orchestration Svc-commandes Jaeger keycloak	Unless-stopped	-	-
prometheus	-	Unless-stopped	-	-
grafana	Prometheus	Unless-stopped	-	-
postgres_kc	-	Unless-stopped	-	-
kafka kafka2 kafka3	keycloak	Unless-stopped	Kafka-broker-api-versions	Interval: 10s Timeout: 10s Retries: 10 Start: -
jaeger	-	Unless-stopped	-	-

keycloak	Postgres_kc	Unless-stopped	-	-
svc_orchestration	Mysql healthy Kafka healthy	Unless-stopped	-	-
svc_clients	Mysql healthy Redis healthy Kafka healthy	Unless-stopped	-	-
svc_commandes	Mysql healthy Redis healthy Kafka healthy	Unless-stopped	-	-
svc_catalogue	Mongo healthy Redis healthy Kafka healthy	Unless-stopped	-	-
svc_facturation	Mysql healthy Mongo healthy Redis healthy Kafka healthy	Unless-stopped	-	-
svc_audit	Kafka healthy	Unless-stopped	-	-
svc_lignes (usage_api)	HLR/db HLR/webui	-	-	-

7.2 Tracing distribué

Afin d'assurer l'observabilité des traitements distribués, les microservices sont instrumentés avec OpenTelemetry. L'objectif est de suivre une même opération métier lorsqu'elle traverse plusieurs services, autant par des appels REST synchrones que par des événements Kafka.

Lorsqu'une requête entre dans le système par l'API Gateway, un contexte de traçage est créé ou repris à partir des en-têtes standards W3C, notamment traceparent. Ce contexte contient principalement un `traceId`, qui identifie l'opération distribuée complète, ainsi qu'un `spanId`, qui représente une étape particulière de cette opération.

Le contexte est ensuite propagé lors des appels REST entre les services. Chaque microservice crée un nouveau `span` enfant tout en conservant le même `traceId`. Il devient ainsi possible de reconstruire le chemin complet d'une requête à travers l'API Gateway et les différents services impliqués.

Pour les communications asynchrones, le contexte de traçage est ajouté aux en-têtes du message publié dans Kafka. Le `traceId` est également conservé dans les métadonnées de l'enveloppe d'événement afin de faciliter le diagnostic et l'inspection manuelle des messages. Kafka permet d'associer des en-têtes aux messages, ce qui convient à la propagation d'informations techniques sans modifier leur charge utile métier.

Une enveloppe d'événement contient notamment les informations suivantes :

```
{
  "eventId": "6d44e87b-4c0b-46d4-89c7-cf6f95b68048",
  "eventType": "ServiceProvisionOk",
  "eventVersion": "1.0",
  "occurredAt": "2026-08-05T15:30:00Z",
  "producer": "svc-lignes",
  "correlationId": "order-8402",
```



```

"causationId": "event-7421",
"traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
"aggregateId": "line-3321",
"idempotencyKey": "provision-order-8402",
"payload": {
  "orderId": "8402",
  "lineId": "3321",
  "status": "PROVISIONED"
}
}

```

Les métadonnées remplissent les responsabilités suivantes :

- eventId : identification unique et déduplication;
- eventType et eventVersion : désérialisation et versionnement;
- correlationId : regroupement des événements d'une même saga;
- causationId : identification de l'événement ayant déclenché celui-ci;
- traceId : corrélation avec la trace OpenTelemetry;
- aggregateId : partitionnement Kafka et maintien de l'ordre local;
- idempotencyKey : prévention des effets métier répétés.

Lorsqu'un consommateur reçoit l'événement, il extrait le contexte OpenTelemetry des en-têtes du message et crée un nouveau span lié au span de publication. Cette propagation permet de conserver une trace logique unique, même lorsque le traitement est différé ou exécuté par plusieurs consommateurs.

Les traces sont exportées vers un collecteur OpenTelemetry, qui centralise les spans produits par les microservices. Le collecteur transmet ensuite les données au backend de traçage Jaeger afin de permettre leur consultation et leur analyse. Le gabarit du projet prévoit précisément OpenTelemetry et Jaeger pour l'observabilité de bout en bout.

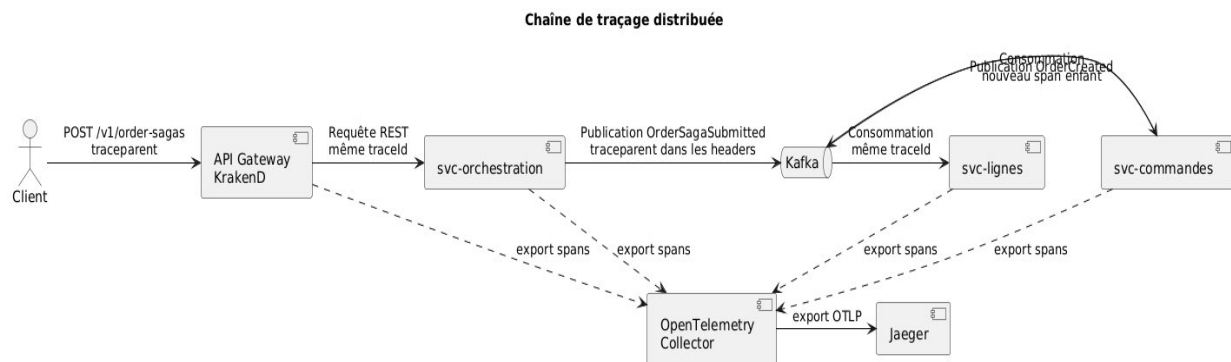


Figure 18 Chaîne de traces distribuées

Une stratégie d'échantillonnage limite le volume de données généré. Dans les environnements de développement et de démonstration, un taux de 100 % est utilisé afin de rendre tous les scénarios observables. En production, ce taux pourrait être réduit, tout en conservant systématiquement les traces associées aux erreurs, aux compensations, aux délais élevés et aux événements envoyés dans une DLQ.

La corrélation entre les traces et les journaux applicatifs est réalisée en ajoutant le traceId, le spanId, le correlationId et l'eventId dans chaque journal structuré :

```

{
  "timestamp": "2026-08-05T15:30:02.418Z",
  "level": "INFO",
  "service": "svc-lignes",

```



```

"message": "Provisionnement de la ligne terminé",
"traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
"spanId": "00f067aa0ba902b7",
"correlationId": "order-8402",
"eventId": "6d44e87b-4c0b-46d4-89c7-cf6f95b68048",
"orderId": "8402",
"lineId": "3321",
"status": "PROVISIONED"
}

```

Cette corrélation permet, à partir d'une erreur observée dans les logs, de retrouver directement la trace distribuée correspondante. Inversement, une trace affichée dans Jaeger permet de repérer les journaux générés par chacun des services impliqués.

Les captures démontrant la propagation du `traceId`, l'enchaînement des spans et la corrélation avec les journaux sont présentées au §10.4.

7.3 API Gateway & load balancing

Renvoi Phase 1 : l'API Gateway (routage, en-têtes/clé API, CORS) et le load balancing (NGINX/HAProxy/Traefik, N = 1..4) restent en place. Documentez ici uniquement les ajustements liés à l'événementiel (ex. exposition d'un endpoint de statut de saga).

[À compléter — ajustements Gateway/LB spécifiques à la Phase 2, le cas échéant.]

Les mécanismes d'API Gateway et de répartition de charge mis en place durant la Phase 1 sont conservés. L'API Gateway demeure le point d'entrée unique pour les clients externes et continue d'assurer le routage, la gestion des en-têtes, la validation des clés API, la limitation du débit ainsi que la configuration CORS.

L'architecture événementielle ne remplace pas les interfaces REST exposées aux clients. Les événements sont principalement utilisés pour les communications internes et pour l'exécution asynchrone des processus métier. Le code conserve d'ailleurs des adaptateurs entrants REST et Kafka distincts, conformément à la structuration hexagonale.

Exposition du statut d'un traitement asynchrone

Lorsqu'une requête déclenche une saga, le résultat final n'est pas nécessairement disponible immédiatement. L'API retourne donc une réponse `202 Accepted` accompagnée de l'identifiant de la saga et de l'adresse permettant d'en consulter l'état.

Requête :

POST /v1/order-sagas

Content-Type: application/json

Idempotency-Key: order-request-8402

traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01

Réponse :

HTTP/1.1 202 Accepted

Location: /v1/order-sagas/order-8402

Retry-After: 2

```

{
  "sagaId": "order-8402",
  "status": "SUBMITTED",

```

```
"message": "La commande a été acceptée pour traitement.",
"statusUri": "/v1/order-sagas/order-8402",
"traceId": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

Le client peut ensuite consulter un endpoint de statut exposé par l'API Gateway : GET /v1/order-sagas/order-8402

Exemple de réponse :

```
{
  "sagaId": "order-8402",
  "orderId": "8402",
  "status": "PROVISIONING",
  "currentStep": "SERVICE_PROVISION",
  "startedAt": "2026-08-05T15:30:00Z",
  "updatedAt": "2026-08-05T15:30:03Z",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

Les états possibles peuvent notamment être :

- SUBMITTED
- VALIDATING
- VALIDATED
- PROVISIONING
- INVOICING
- COMPLETED
- COMPENSATING
- CANCELLED
- FAILED
- EXPIRED

L'API Gateway route cet endpoint vers `svc-orchestration`, qui conserve ou reconstruit l'état courant de la saga à partir des événements reçus. Le dépôt contient déjà un contrôleur de saga et un dépôt de résultats de provisionnement dans ce service.

Propagation du contexte

La Gateway conserve ou génère les en-têtes de traçage et de corrélation suivants : `traceparent`, `tracestate`, `baggage`, `X-Correlation-ID`, `X-Request-ID`, `Idempotency-Key`. ``traceparent``, ``tracestate`` et ``baggage`` servent au traçage OpenTelemetry. ``X-Correlation-ID`` identifie la saga, tandis que ``Idempotency-Key`` permet de reconnaître la répétition d'une même commande client.

La Gateway ne communique pas directement avec Kafka. Elle route les requêtes HTTP vers les adaptateurs entrants des services applicatifs. Ceux-ci invoquent ensuite leurs cas d'utilisation et publient les événements à travers leurs adaptateurs sortants et l'outbox.

Répartition de charge

La répartition de charge de la Phase 1 demeure applicable aux requêtes REST. Lorsque plusieurs instances d'un microservice sont démarrées, le répartiteur distribue les requêtes HTTP entre les instances disponibles.

Pour les traitements Kafka, la répartition est effectuée par les groupes de consommateurs. Les instances appartenant au même service utilisent le même ``group.id``; Kafka attribue alors chaque partition à une seule instance du groupe à la fois. Cette approche permet de répartir les événements

entre les instances et de reprendre automatiquement les partitions lorsqu'un consommateur devient indisponible.

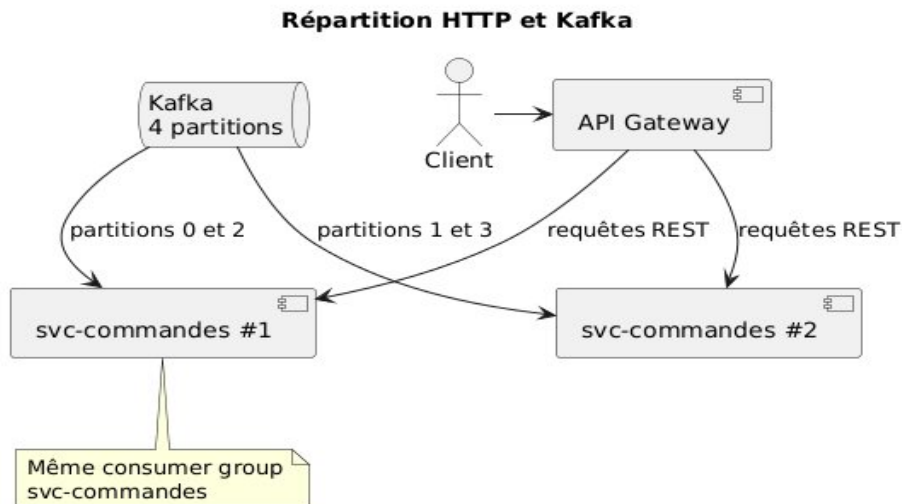


Figure 19 Répartition de charge événementielle versus REST

Le nombre maximal de consommateurs actifs d'un même groupe est limité par le nombre de partitions du topic. Avec quatre partitions, jusqu'à quatre instances peuvent traiter les messages en parallèle; toute instance supplémentaire demeure en attente jusqu'à une réattribution.

Les consommateurs restent idempotents, puisque la livraison `at-least-once` peut provoquer la réception répétée d'un événement après une panne ou avant la validation de l'offset.

Frontière entre la Gateway et le système événementiel

L'API Gateway reste limitée aux responsabilités liées au trafic HTTP externe :

- authentification et validation des clés;
- routage;
- CORS;
- limitation du débit;
- délais d'attente;
- répartition des requêtes;
- propagation des en-têtes de corrélation.

Elle ne contient aucune logique métier et ne coordonne pas directement la saga. Les décisions métier et les changements d'état demeurent dans les microservices. Kafka reste responsable du transport des événements internes.

Cette séparation évite de transformer la Gateway en orchestrateur central et maintient l'isolation entre les responsabilités d'infrastructure et le domaine.

Ajustements apportés en Phase 2

Les ajustements spécifiques à la Phase 2 sont :

1. l'exposition d'un endpoint de consultation du statut des sagas
2. l'utilisation de réponses `202 Accepted`

3. la propagation des en-têtes OpenTelemetry et des identifiants de corrélation
4. l'utilisation de `Idempotency-Key` pour les commandes
5. la réduction des délais HTTP, puisque la requête n'attend plus la fin de la saga
6. la prise en compte des groupes de consommateurs et du nombre de partitions
7. l'ajout de contrôles de santé pour Kafka, les consommateurs et les publishers outbox
8. la conservation des consommateurs idempotents malgré la répartition de charge

8. Concepts transversaux

8.1 Outbox pattern

Outbox Table (Entity-Relationship Diagram)

outbox
• id : Integer «PK» «autoincrement»
• topic : String(100) • aggregate_type : String(100) • aggregate_id : BigInteger • event_type : String(100) • payload : JSON • created_at : DateTime «default: CURRENT_TIMESTAMP» • published : Boolean «default: FALSE»

Figure 20 Schéma de la table Outbox

Afin d'assurer une publication fiable des événements, chaque microservice utilise le patron Outbox. Lorsqu'une transaction métier est validée, les modifications de la base de données et l'événement à publier sont enregistrés de façon atomique dans la même transaction. Un processus de publication dédié lit ensuite la table Outbox et diffuse les événements vers Kafka.

Cette approche élimine le risque qu'une transaction soit validée sans que l'événement correspondant soit publié, ou inversement. Les événements demeurent dans l'Outbox jusqu'à leur publication réussie, puis sont marqués comme traités. L'Outbox constitue ainsi un mécanisme fiable de livraison tout en conservant une trace des événements publiés.

8.2 Saga chorégraphiée & compensation

Les transactions distribuées sont implémentées sous la forme d'une saga chorégraphiée. Chaque service est responsable de réagir aux événements qu'il consomme et de publier les événements décrivant le résultat de son traitement. Aucun orchestrateur central ne coordonne les échanges.

Cette approche réduit le couplage entre les services, facilite leur évolution indépendante et améliore la résilience du système. En cas d'échec, les événements de compensation permettent d'annuler les opérations déjà réalisées afin de rétablir un état cohérent. La progression de la saga est observable grâce aux événements publiés sur Kafka et au tracing distribué.

Lors d'une compensation, l'événements s'enfiles comme dans un cas nominal, sauf que l'objectif et de défaire les opérations effectuées par la saga nominale qui est tombée en erreur.

Dans tous les cas, l'état terminal de la saga est atteint. Certaines sagas peuvent ne pas répondre en temps voulu (p.ex pour l'activation d'une ligne ou la portabilité d'un numéro). Alors, un mécanisme doit ramener le système à l'état d'avant la saga par compensation. Le délai d'attente de traitement est de 20 minutes.

8.3 Idempotence des consommateurs & déduplication

Le système applique le principe d'idempotence afin de supporter une livraison at-least-once des événements. Chaque commande et chaque événement possèdent un identifiant de corrélation ainsi qu'une idempotencyKey permettant d'identifier de façon unique une opération métier.

Avant d'exécuter une action, un consommateur vérifie si l'événement a déjà été traité. Si un doublon est détecté, l'événement est ignoré sans produire de nouvel effet métier. Cette stratégie garantit qu'une même opération (activation de ligne, portabilité, facturation, etc.) n'est exécutée qu'une seule fois, même en présence de retransmissions.

La clé d'idempotence est aussi utilisée comme traceId pour corréler les événements et pouvoir les observer dans Jaeger.

8.4 Ordre & garanties de livraison

Les communications événementielles reposent sur une garantie at-least-once. Un événement peut donc être reçu plusieurs fois, mais ne doit jamais être perdu. L'association du patron Outbox, de Kafka et des consommateurs idempotents permet d'obtenir un effet fonctionnel équivalent à un traitement unique et d'atteindre l'*exactly-once*.

L'ordre des événements est conservé à l'intérieur d'une même partition Kafka grâce au partitionnement utilisant la clé de corrélation de la transaction. Les événements d'une même saga sont ainsi consommés dans leur ordre de publication.

8.5 Cohérence à terme (eventual consistency)

L'architecture événementielle privilégie la cohérence à terme plutôt qu'une cohérence immédiate. Après la création d'une commande, les différents microservices mettent progressivement leur état à jour au rythme des événements de la saga.

Durant cette période, certains services peuvent temporairement présenter des informations différentes. Une fois tous les événements consommés, le système converge vers un état cohérent. En cas d'échec, les mécanismes de compensation assurent également cette convergence.

8.6 Observabilité événementielle (4 Golden Signals + tracing)

Logs structurés + métriques Prometheus + tracing distribué + dashboards Grafana. Ajoutez aux 4 Golden Signals les métriques propres au broker (débit, latence de consommation, lag).

Il ne nous a pas été possible d'observer le bus événementiel car l'implémentation des événements n'est pas complétée.

Signal / métrique	Indicateur	Cible
Latence	P95 (cible) / P99 (queue, secondaire)	$P95 \leq 250$ ms (cible Phase 2 ; 500 ms en Phase 1)
Trafic	RPS + événements/s produits/consommés	≥ 1000 opérations/s au point d'entrée; débit consommé $\geq$ débit produit sur une fenêtre de 5 min.
Erreurs	4xx/5xx + messages en DLQ	< 1 % de 5xx en charge nominale; 0 message en DLQ dans le scénario nominal; alerte dès $DLQ > 0$.

Saturation	CPU/RAM/threads + lag de consommation	CPU < 80 %, RAM < 85 %, lag < 100 messages et retard de consommation < 5 s en régime nominal.
Tracing	Trace complète d'un flux (traceld)	Bout-en-bout couvert

8.7 Gestion d'erreurs événementielle (DLQ, retries, backoff)

Lorsqu'un traitement échoue à cause d'une erreur temporaire (service indisponible, panne réseau, etc.), le consommateur effectue plusieurs tentatives de traitement (retries) espacées par un délai croissant (exponential backoff) afin d'éviter de surcharger les services.

Si l'événement demeure impossible à traiter après le nombre maximal de tentatives, il est redirigé vers une Dead Letter Queue (DLQ). Les événements présents dans cette file peuvent être analysés par les équipes d'exploitation, corrigés puis rejoués sans interrompre le fonctionnement normal du système.

8.8 Versionnement des schémas d'événements

Les contrats d'événements sont documentés à l'aide d'AsyncAPI et versionnés afin de préserver la compatibilité entre producteurs et consommateurs. Chaque événement possède un schéma clairement défini et évolue selon des règles de compatibilité ascendante.

L'ajout de nouveaux champs optionnels est privilégié afin d'éviter de casser les consommateurs existants. Les schémas sont centralisés dans le registre de schémas Confluent, ce qui facilite leur validation et leur évolution au fil des versions.

Le versionnement des schémas est v1 pour tous événements de la phase II. AsyncAPI est dans le dossier docs/ du répertoire du projet principal (cantelcox) et nous l'ajoutons en Annexe E.

8.9 Intégration fiable des systèmes fournis (HLR/HSS & PortabilityHub)

Les appels aux systèmes fournis sont des points d'intégration critiques dans la saga. Ils sont encapsulés dans des adapters + couche anticorruption (ACL) et traités comme des étapes susceptibles d'échouer (donc compensables).

Le HLR/HSS et le PortabilityHub sont des systèmes externes imposés par l'énoncé et sont intégrés au moyen d'adaptateurs respectant l'architecture hexagonale. Les services métier ne dépendent donc pas directement des interfaces techniques de ces fournisseurs.

Le HLR/HSS est utilisé pour le provisionnement et l'activation des lignes mobiles, tandis que le PortabilityHub gère les demandes de portabilité des numéros. Les deux intégrations utilisent des clés d'idempotence, participent à la saga chorégraphiée et publient les événements nécessaires à la poursuite ou à la compensation des transactions. Les échanges sont entièrement instrumentés afin d'assurer la traçabilité, l'audit et le suivi distribué des opérations.

Préoccupation	Mise en œuvre attendue
Isolation (ACL)	Adapters HlrProvisioningAdapter et PortabilityHubAdapter + ACL. Les DTO externes sont traduits vers MobileLine, Msisdn, Supi et PortabilityStatus; aucun modèle free5GC/Hub ne traverse le domaine.
Idempotence	HLR : clé orderId:MSISDN et retour idempotent de la ligne existante.

	Hub : idempotencyKey de la saga sur port-in/out et déduplication du callback par eventId.
Résilience	Timeout HTTP configuré, reprise limitée des erreurs transitoires avec backoff, rafraîchissement unique du jeton free5GC; circuit breaker recommandé avant production.
Échec → compensation	Échec HLR → ServiceProvisionKo/ActivationÉchouée; REJECTED ou EXPIRED du Hub → PortabilitéÉchouée, libération des réservations, restauration du stock et annulation.
Asynchronisme du Hub	ACK immédiat, puis callback corrélé par orderId + idempotencyKey. La saga reste PENDING jusqu'à PORTED/REJECTED; l'échéance de fenêtre produit EXPIRED.
Observabilité	Traceparent/traceld/correlationId propagés; métriques de latence, taux d'erreur, timeout, retry et résultat final par système externe. Captures non jointes.

9. Décisions d'architecture (ADR)

Objectif Arc42 — Consigner les décisions structurantes de la Phase 2 au format ADR (statut, contexte, décision, conséquences), chacune traçable à une exigence de l'énoncé

ADR-004 — Adoption de l'architecture événementielle (chorégraphie)

Champ	Contenu
Statut	Accepté
Contexte	Le traitement d'une commande de télécommunication implique plusieurs microservices autonomes, notamment les services de commande, d'inventaire, de facturation et d'audit, ainsi que les systèmes externes HLR/HSS et PortabilityHub. Une orchestration synchrone centralisée, tel que dans la phase 1 crée un point de couplage important et un point unique de défaillance. La phase 2 demande de favoriser les communications asynchrones et de rendre le système plus résilient aux indisponibilités temporaires.
Décision	Adopter une architecture événementielle chorégraphiée. Chaque microservice publie des événements représentant les changements de son domaine et réagit uniquement aux événements auxquels il est abonné. Aucun service central ne contrôle l'ensemble du processus. Les événements sont publiés dans Kafka et sont versionnés au moyen d'un champ de version dans leur enveloppe.
Conséquences	positives – réduction du couplage entre les services – meilleure autonomie des équipes et des microservices – traitement asynchrone – meilleure tolérance aux pannes temporaires – ajout plus simple de nouveaux consommateurs négatives – débogage distribué plus complexe – cohérence éventuelle des données – cheminement d'une transaction plus difficile à suivre – nécessité d'assurer l'idempotence
Exigence tracée	Sections §4.1, §6.1, §6.2, §8.2

ADR-005 — Outbox pattern pour la publication fiable

Champ	Contenu
Statut	Accepté
Contexte	Une opération métier peut réussir dans la base de données locale alors que la publication de l'événement correspondant dans Kafka échoue. L'inverse peut également se produire si l'événement est publié avant l'annulation de la transaction locale. Comme une transaction distribuée entre la base de données et Kafka n'est pas souhaitable, il faut empêcher la perte d'événements et limiter les incohérences.
Décision	Utiliser le pattern Transactional Outbox dans chaque microservice qui

	produit des événements. La modification métier et l'ajout de l'événement dans une table outbox sont réalisés dans la même transaction locale. Un processus de relais lit ensuite les événements non publiés, les transmet à Kafka, puis les marque comme publiés. Chaque entrée contient notamment un identifiant d'événement unique, le type, la version, l'identifiant d'agrégat, l'identifiant de corrélation, la charge utile et la date de création.
Conséquences	positives – aucune perte d'événement – absence de transaction distribuée – possibilité de reprendre la publication après une panne – meilleure traçabilité négatives – ajout d'une table et d'un processus de relais dans les services producteurs – délai possible entre la transaction et la publication – nettoyage périodique requis – possibilité de doublons, puisque la livraison est garantie au moins une fois – les consommateurs doivent donc être idempotents
Exigence tracée	Sections §4.3, §6.3, §8.1

ADR-006 — Saga chorégraphiée & compensation

Champ	Contenu
Statut	Accepté
Contexte	L'implémentation d'une saga chorégraphiée éprouve le besoin d'une garantie que les transactions métiers à travers les services restent cohérentes dans le cas d'une défaillance dans le système. Les simples "roll-backs" ne sont pas possible dans une architecture en microservice où chaque contexte est indépendant et possède sa propre base de donnée.
Décision	Implémentation d'un système de compensation dans la saga chorégraphiée
Conséquences	positives – Faible couplage entre les microservices – Chaque service déclenche sa propre compensation face à un événement d'échec, ils n'ont pas besoin d'attendre l'ordre d'un orchestrateur – L'échec d'un service n'affecte pas les autres services négatives – Nécessite un mécanisme de journalisation bien défini pour faire le tracage et la reconstruction de l'état du système – Complexité lors du débogage d'échec dans la chaîne d'événements – Idempotence importante à retenir compromis –

Exigence tracée	Sections §4.4; §6: Saga chorégraphiée & compensation
-----------------	--

ADR-007 — Choix du broker de messages

Champ	Contenu
Statut	Accepté
Contexte	La conversion du système du BSS d'une saga orchestrée à une saga chorégraphiée nécessite de faire une décision sur le choix d'event broker de message entre Kafka, RabbitMQ et NATS. Dans ce choix d'event broker, il est important de considérer comment l'implémentation de l'event broker affectera l'architecture de notre solution, comment elle nous permettra de faire évoluer notre système et comment elle affecte la performance du BSS.
Décision	Adoption d'une Saga Chorégraphiée basée sur Kafka en tant qu'event broker pour coordonner les transactions distribuées dans notre architecture microservices. Cette décision s'aligne avec l'évolution vers une architecture event-driven et les meilleures pratiques cloud-native: séparation en microservices, scalabilité et résilience. De plus, il est choisi d'avoir un canal (<i>topic</i>) événementiel par état de la chorégraphie, pour un traitement granulaire et isolé des changements d'état. Une grappe de trois serveurs (engin KRaft) et quatre partitions par topics est choisi pour l'implémentation.
Conséquences	positives – Scalabilité horizontale – Peut gérer une haute quantité de messages à la fois – Consommateur d'events indépendants – Haute résilience négatives – compromis –
Exigence tracée	Sections §4.2, §5.3, §8.4

ADR-008 — Intégration des systèmes fournis (HLR/HSS & PortabilityHub) via adaptors/ACL

Champ	Contenu
Statut	Acceptée
Contexte	Le cahier de projet impose l'utilisation du HLR/HSS et du PortabilityHub fournis par l'encadrement. Ces systèmes externes utilisent leurs propres API et modèles de données, qui ne doivent pas être directement exposés au domaine métier. L'architecture hexagonale retenue pour le projet vise à préserver l'indépendance du domaine tout en facilitant l'évolution des intégrations externes.
Décision	L'intégration avec le HLR/HSS et le PortabilityHub est réalisée au moyen d'adaptors situés dans la couche Infrastructure (Ports & Adapters). Une

	Anti-Corruption Layer (ACL) traduit les modèles de données et les protocoles des systèmes externes vers les objets métier internes. Les appels au HLR/HSS demeurent synchrones puisqu'ils correspondent à des opérations de provisionnement réseau, tandis que le PortabilityHub est intégré selon son fonctionnement asynchrone (ACK suivi d'un callback contenant le résultat final). Les réponses des deux systèmes sont converties en événements métier publiés sur Kafka afin de poursuivre ou compenser la saga.
Conséquences	Positives – Le domaine métier reste indépendant des APIs – Un changement d'API ou de fournisseur n'impacte que les adaptateurs. – Les modèles internes demeurent cohérents et indépendants des formats externes. – Les échanges peuvent être instrumentés (traces, métriques, journalisation) sans modifier le domaine. – Les appels sont rendus idempotents grâce aux clés de corrélation utilisées par la saga. Négatives – Ajout d'une couche de traduction augmentant la complexité du code. – Maintenance supplémentaire des adaptateurs et des règles de conversion. – Les évolutions des API externes nécessitent une mise à jour des Anti-Corruption Layer (ACL). Compromis –
Exigence tracée	Sections §4.6, §6.4, §8.9.

ADR-009 — Utilisation de AsyncAPI

Champ	Contenu
Statut	Accepté
Contexte	L'échange de message structuré sous forme d'événements est un problème complexe et on souhaite utiliser un standard ouvert nous permettant de maximiser la qualité du logiciel.
Décision	Il est choisi d'utiliser AsyncAPI.com pour s'accoter au standard industriel. On y définit les opérations, les messages et tous les canaux. Cette définition sert l'implémentation de manière cohérente et sécuritaire.
Conséquences	positives – Standard complet et bien documenté – Agnostique du protocole, permet une plus grande interopérabilité – Outils de conception et de génération de code intégré – Clé de corrélation à même l'en-tête des messages pour faciliter la traçabilité négatives

	– Courbe d'apprentissage – Temps de standardisation compromis – Y aller mais le plus simplement possible pour commencer
Exigence tracée	Sections §5.3, §8.8, Annexe A

ADR-010 — Garantie de publication des messages et cohérence des noeuds

Champ	Contenu
Statut	Accepté
Contexte	La publication de messages au bus Kafka peut échouer. Dans ce cas, il est nécessaire de s'assurer de la publication d'un message par un <i>Publisher</i> Kafka.
Décision	Il est choisi de gérer la garantie de publication avec les options de confirmation (<i>acks</i>) de Kafka en utilisant l'option <i>acks=all</i> .
Conséquences	positives – La cohérence de la grappe Kafka est assurée par une réponse positive impliquant que tous les noeuds actifs ont répliqué la publication de l'événement – Possibilité de faire disjoncter le service qui ne reçoit pas de confirmation pendant un certain temps – Cohérence de tous les noeuds assurée par le mécanisme négatives – Un peu de latence induite pour que tous les noeuds actifs confirment l'écriture, au profit d'une meilleure cohérence compromis – Intégrer le principe dans tous les services ne devrait pas induire trop de latence alors on y va.
Exigence tracée	Sections §6.3, §8.3, §8.4, §8.5

ADR-011 — Cohabitation REST synchrone ↔ événementiel

Champ	Contenu
Statut	Accepté
Contexte	Durant la transition deux systèmes vivent en même temps. On doit déterminer comment ils existent dans le système.
Décision	Il est choisi de maintenir la fonctionnalité synchrone telle quelle par l'utilisation de l'API v1 du service d'orchestration. La version asynchrone est assurée par l'API v2 en mode chorégraphie.
Conséquences	positives – Rétrocompatibilité – Compensation complète si Kafka meurt négatives

	– La charge du traitement synchrone va polluer celle de l'asynchrone, réduisant ainsi les bénéfices de l'architecture compromis – Dédier une partie du système à un mode synchrone, avec des répartiteurs de charge en conséquence. Tous les appels v1 sont envoyés sur ces répartiteurs – Laisser les services dans la partie synchrone écouter quand même sur kafka, ils pourront répondre s'ils le peuvent – Dédier une autre partie du système au mode asynchrone, pour la v2
Exigence tracée	Sections §4.5, §6.1, §6.4, §7.3

10. Exigences de qualité — gains mesurés

10.1 Cibles NFR & résultats

Malheureusement, l'implémentation n'a pas été complétée pour l'événementiel mais les cibles ont été définies. On s'attend toutefois à des gains significatifs mais nous ne sommes pas en mesure de pouvoir le démontrer.

Indicateur	Cible	Mesuré	Verdict
Latence P95 (cible)	≤ 250 ms (500 ms en Phase 1)	Non mesuré dans les artefacts fournis	Écart — campagne à exécuter
Latence P99 (queue, secondaire)	≤ 1 s; surveillance du temps d'attente et du lag	Non mesuré	Non démontré
Débit	≥ 1000 ops/s (600 en Phase 1)	Non mesuré	Non démontré
Disponibilité	99 % (95 % en Phase 1)	fenêtre représentative	Non démontré
Convergence saga	≤ 5 s nominal; ≤ 30 s après reprise	E2E fonctionnel réussi; durée non consignée	Fonctionnel, NFR non démontré

10.2 Résultats des campagnes de charge

Les campagnes de charge sont les mêmes que celles de la phase I et sont prêtes à être lancées, mais l'implémentation n'est pas complétée alors nous ne pouvons pas démontrer de résultats.

10.3 Comparatif synchrone REST ↔ événementiel

Scénario	Synchrone (Phase 1)	Événementiel (Phase 2)	Gain / écart
Prise de commande	E2E Phase 1 fonctionnel; P95 non consignée	E2E Phase 2 fonctionnel; P95 non consignée	Gain chiffré non démontré; réponse 202 réduit toutefois le temps bloqué côté client.
Pic de charge	Débit/erreurs non joints	Débit/erreurs/lag non joints	Comparaison impossible sans campagne identique.
Panne d'un consommateur	L'appel synchrone échoue ou bloque la chaîne.	L'événement reste dans Kafka/outbox et peut être repris; reprise chiffrée non testée.	Résilience attendue supérieure, mais preuve de chaos test à ajouter.

10.4 Observabilité : tracing, lag broker & 4 Golden Signals

Le code disponible instrumente svc-orchestration avec OpenTelemetry et export OTLP vers Jaeger; svc-lignes expose /metrics et les services utilisent des métriques Prometheus. La corrélation attendue repose sur traceId dans les logs, l'enveloppe et les en-têtes Kafka.

Malheureusement, l'implémentation n'a pas été complétée et nous n'avons pas de traces à fournir pour l'observabilité événementielle. Cependant, l'observabilité de la phase I est accessible en phase II.

Les 4 signaux sont quant à eux récupérés sur toutes les instances via Grafana : latence, débit du trafic, ressources (saturation) et erreurs. C'est la même observabilité des métriques que la phase I.

10.5 Load balancing & caching

Renvoi Phase 1 — reconduits en Phase 2. Reportez ici les mesures mises à jour si elles changent sous charge événementielle.

Dans la phase II, des répartiteurs de charge ont été ajoutés devant les services et API gateway de la phase I, pour rattraper la dette technique à ce niveau. Toutefois, nous n'avons pas été mesure de compléter l'implémentation et ainsi n'avons pas de mesures à présenter pour les tests de charge faits spécifiquement pour la partie événementielle en phase II.

11. Risques et dette technique

Objectif Arc42 — Identifier les risques propres à l'événementiel et la dette technique, avec leurs mesures d'atténuation.

Risque / dette	Impact	Probabilité	Atténuation
Livraison dupliquée	Double effet (paiement, commandes, etc.)	Moyen	Idempotence + déduplication (eventId) + outbox pattern.
Désordre des événements	État incohérent (ex: solde mis à jour avant confirmation de commande)	Élevé	Partitionnement par clé d'agrégat
Message empoisonné	Blocage du flux	Moyen	DLQ + retries/backoff
Cohérence à terme mal comprise	Attente client, perte d'information	Moyen	Statut de saga + documentation
Complexité architecturale	Coûts de développement élevés, forte expertise	Élevé	Documentation + formation

12. Glossaire

Objectif Arc42 — Définir le langage ubiquitaire et les termes propres à l'architecture événementielle pour une compréhension commune.

12.1 Termes événementiels

Terme	Définition	Remarque
Événement de domaine	Fait métier passé, immuable, publié par un service	Nommé au passé
Commande	Intention d'action déclenchant un traitement	Peut échouer
Saga chorégraphiée	Transaction distribuée sans orchestrateur, par réactions en chaîne	§4.4
Compensation	Action inverse défaisant une étape en cas d'échec	§6.2
Outbox pattern	Publication atomique via une table tampon + relais	§8.1
Idempotence	Rejouer un traitement sans changer le résultat	§8.3
At-least-once	Chaque message livré au moins une fois (donc possibles doublons)	§8.4
Exactly-One	Propriété d'une transaction qui est garantie de s'effectuer une fois et qu'une seule	
DLQ (Dead Letter Queue)	File des messages non traitables	§8.7
Eventual consistency	Cohérence atteinte à terme après propagation	§8.5
Tracing distribué	Suivi d'un flux via un traceId propagé	§7.2
HLR / HSS (fourni)	Registre d'abonnés du réseau mobile ; provisionne les lignes (MSISDN) — système fourni, intégration obligatoire	§4.6 ; §6.1
PortabilityHub (fourni)	Hub de portabilité des numéros (port-in / port-out) — système fourni, intégration obligatoire	§4.6 ; §6.4
Adapter / ACL	Adapter (ports/adapters) + couche anticorruption isolant un système externe du domaine	§4.6 ; §8.9
Topic	Canal de messagerie événementiel	

12.2 Acronymes

Acronyme	Signification
ADR	Architecture Decision Record
CDC	Change Data Capture

DLQ	Dead Letter Queue
OTel	OpenTelemetry
NFR	Non-Functional Requirement (exigence non fonctionnelle)
HLR / HSS	Home Location Register / Home Subscriber Server
MSISDN	Mobile Station ISDN Number (numéro de la ligne mobile)
ACL	Anti-Corruption Layer (couche anticorruption)
MFA / OTP	Authentification multifacteur / mot de passe à usage unique
DDD	Domain-Driven Design
CRTC	Conseil de la radiodiffusion et des télécommunications canadiennes
TTL	Time-To-Live

12.3 Termes métier

On reprend ici le glossaire de la phase I que l'on bonifie à la fin.

Terme	Définition	Bounded context
API Gateway	Passerelle d'accès publique au système	CanTelcoX
MSISDN	Identifiant de ligne mobile	Lignes
ProductOffering	Ensemble services et limites d'usage applicable sur une ligne avec ou sans produit (p.ex. téléphone) qu'un abonné peut obtenir à un prix mensuel. C'est un forfait.	Catalogue
Ligne	Association d'un MSISDN à un profil utilisateur	Lignes
Service	Service activable sur une ligne	Lignes
Usage	Quantification de l'utilisation d'un service à un moment donné, à la seconde près	Lignes
Tarification	Calcul des montants excédentaires pour utilisation de services en-dehors des limites du forfait	Facturation
Abonné	Client pouvant souscrire à un produit du catalogue	CanTelcoX
Attachment	Message lié à un événement du système	Audit
Notification	Alerte ou avis propre à un événement du système	Audit

Terme	Définition	Bounded context
Topic	Regroupement d'événement dans un sujet	Audit
Event	Événement du système	Audit
AntiFraude	Service mettant en application des principes et processus d'inspection, de validation et de réaction à la fraude.	Audit
BillItem ou BillLine	Ligne de facture client	Facturation
RatedProductUsage	Utilisation d'un produit à l'usage mesuré, comme une ligne p. ex.	Facturation
BillingAccount	Compte de facturation client	Facturation
BillCycle	Cycle de facturation client	Facturation
Usage	Données d'utilisation de service mobile	Facturation
CustomerBill	Facture client	Facturation
ProductOffering	Offres de produit, définit les caractéristiques de prix et de description d'une offre	Commande
ProductCatalog	Catalogue des produits	Commande
Product	Un produit est lié à une offre de produit et à une ligne de commande	Commande
ProductOrderItem	C'est une ligne de commande liée à un produit et à une comande	Commande
OrderStatus	Les états d'une commande	Commande
ProductOrder	La commande, par un client, d'un ou plusieurs produits, dont une ligne mobile	Commande
Characteristic	Caractéristiques descriptives de produit	Commande
Feature	Capacités d'un service	Lignes
Service	Désigne un service de ligne mobile associé au réseau	Lignes
IdentificationStatus	Statut de l'identification du client pour validation	Clients

Terme	Définition	Bounded context
	des pièces d'identité	
DonneesPersonnelles	Données personnelles du client	Clients
Identité	Identifié propre au client, lié à un compte	Clients
Party	C'est un tiers, soit un client	Client
Portabilité	Action de transférer la disponibilité d'un numéro de ligne mobile d'un fournisseur à un autre	Lignes
Portability Hub	<i>Hub</i> de portabilité, système permettant de gérer la portabilité d'un numéro de ligne mobile entre deux systèmes	Lignes

Annexe A — Catalogue d'événements & schémas

Détaillez chaque événement (enveloppe + payload). Dupliquez le tableau pour chaque événement.

Champ	Contenu
Nom de l'événement	OrderSagaSubmitted
Producteur	svc-orchestration
Consommateurs	svc-audit, svc-commandes
Topic / exchange	saga.order-submitted (<i>clé de partition : orderId ou userId</i>)
Payload (schéma)	Champs: userId (string), timeSubmitted (date-time), order (object). <i>Exemple JSON:</i> { "userId": "USR-123", "timeSubmitted": "2026-08-05T09:00:00Z", "order": { "userId": "USR-123", "items": {} } }
Version / compatibilité	v1
Déclenché par / réactions	La soumission de la saga de commande ; Déclenche la création du brouillon.
Champ	Contenu
Nom de l'événement	OrderDraft
Producteur	svc-commandes
Consommateurs	svc-clients, svc-catalogue, svc-ligne
Topic / exchange	commande.order-draft
Payload (schéma)	Champs: orderState (draft, created, cancelled), timeOccured (date-time), order (object). <i>Exemple JSON:</i> { "orderState": "draft", "timeOccured": "2026-08-05T09:05:00Z", "order": { "userId": "USR-123", "items": {} } }
Version / compatibilité	v1
Déclenché par / réactions	Le système de commandes (panier validé) ; Déclenche les vérifications parallèles (client, offres, service).
Champ	Contenu
Nom de l'événement	BuyerVerified
Producteur	svc-clients
Consommateurs	svc-orchestration
Topic / exchange	client.buyer-verified
Payload (schéma)	Champs: partyRef (string), orderId (integer), validation (OK, KO).

	<i>Exemple JSON:</i> { "partyRef": "PTY-456", "orderId": 1001, "validation": "OK" }
Version / compatibilité	v1
Déclenché par / réactions	Réception de l'événement OrderDraft ; L'orchestrateur enregistre le résultat de la vérification.
Champ	Contenu
Nom de l'événement	OfferingsVerified
Producteur	svc-catalogue
Consommateurs	svc-orchestration
Topic / exchange	catalogue.offerings-verified
Payload (schéma)	Champs: orderId (integer), productOfferings (object). <i>Exemple JSON:</i> { "orderId": 1001, "productOfferings": { "name": "Forfait 5G", "code": "F5G", "offeringType": "service" } }
Version / compatibilité	v1
Déclenché par / réactions	Réception de l'événement OrderDraft ; L'orchestrateur vérifie la disponibilité et validité du panier.
Champ	Contenu
Nom de l'événement	ServiceVerified
Producteur	svc-lignes
Consommateurs	svc-orchestration
Topic / exchange	lignes.service-verified
Payload (schéma)	Champs: orderId (integer), serviceRef (string). <i>Exemple JSON:</i> { "orderId": 1001, "serviceRef": "SRV-789" }
Version / compatibilité	v1
Déclenché par / réactions	Réception de l'événement OrderDraft ; L'orchestrateur confirme l'éligibilité technique de la ligne.
Champ	Contenu
Nom de l'événement	OrderValidationsOk
Producteur	svc-orchestration
Consommateurs	svc-commandes

Topic / exchange	saga.order-validations-ok
Payload (schéma)	Champs: orderId (integer), validation (OK, KO). <i>Exemple JSON:</i> { "orderId": 1001, "validation": "OK" }
Version / compatibilité	v1
Déclenché par / réactions	L'orchestrateur a reçu les confirmations OK de tous les sous-systèmes (Client, Catalogue, Lignes) ; Déclenche la création officielle de la commande.
Champ	Contenu
Nom de l'événement	OrderCreated
Producteur	svc-commandes
Consommateurs	svc-audit, svc-catalogue, svc-orchestration
Topic / exchange	commande.order-created
Payload (schéma)	Champs: orderState (created), timeOccured (date-time), order (object). <i>Exemple JSON:</i> { "orderState": "created", "timeOccured": "2026-08-05T09:10:00Z", "order": { "userId": "USR-123", "items": {} } }
Version / compatibilité	v1
Déclenché par / réactions	Réception de OrderValidationsOk ; Déclenche la diminution des stocks et l'activation des services.
Champ	Contenu
Nom de l'événement	StockDecreased
Producteur	svc-catalogue
Consommateurs	svc-orchestration
Topic / exchange	catalogue.stock-decreased
Payload (schéma)	Champs: orderId (integer), affectedProductOfferings (object). <i>Exemple JSON:</i> { "orderId": 1001, "affectedProductOfferings": { "productOfferingRef": "SIM-CARD", "quantity_after": 450 } }
Version / compatibilité	v1
Déclenché par / réactions	Réception de OrderCreated ; L'orchestrateur est notifié que les stocks physiques ont été réservés/déduits.
Champ	Contenu

Nom de l'événement	DoProvisionActions
Producteur	svc-orchestration
Consommateurs	svc-audit, svc-lignes
Topic / exchange	saga.order-do-provision
Payload (schéma)	Champs: orderId (integer), MSISDN (string). <i>Exemple JSON:</i> { "orderId": 1001, "MSISDN": "+15145551234" }
Version / compatibilité	v1
Déclenché par / réactions	L'orchestrateur, après OrderCreated (et StockDecreased) ; Demande au service des lignes d'activer la connectivité.
Champ	Contenu
Nom de l'événement	ServiceProvisionOk
Producteur	svc-lignes
Consommateurs	svc-orchestration
Topic / exchange	lignes.service-provision-ok
Payload (schéma)	Champs: orderId (integer), validation (OK, KO), service (object). <i>Exemple JSON:</i> { "orderId": 1001, "validation": "OK", "service": { "userId": 123, "orderId": 1001, "MSISDN": "+15145551234", "serviceStatus": "active" } }
Version / compatibilité	v1
Déclenché par / réactions	Succès de l'activation réseau locale ; L'orchestrateur poursuit avec la facturation ponctuelle.
Champ	Contenu
Nom de l'événement	OrderSagaEnded
Producteur	svc-orchestration
Consommateurs	svc-audit, svc-clients
Topic / exchange	saga.order-ended
Payload (schéma)	Champs: status (string), orderId (integer), activation (object), custom erbillId (integer), paymentLink (string). <i>Exemple JSON:</i> { "orderId": 1001, "status": "COMPLETED", "activation": { "status": "active" }, "customerbillId": 5001 }

Version / compatibilité	v1
Déclenché par / réactions	L'orchestrateur a reçu le succès du provisionnement et de la facturation ; Clôture de la saga et information du client.
Champ	Contenu
Nom de l'événement	DoInvoicing (<i>type ponctuel</i>)
Producteur	svc-orchestration
Consommateurs	svc-audit, svc-facturation
Topic / exchange	saga.order-do-invoicing
Payload (schéma)	Champs: orderId (integer), billCycleType (mensuel, ponctuel). <i>Exemple JSON:</i> { "orderId": 1001, "billCycleType": "ponctuel" }
Version / compatibilité	v1
Déclenché par / réactions	Étape de facturation dans la saga de commande pour payer les items non-récurrents ; Déclenche l'émission de la facture côté svc-facturation.
Champ	Contenu
Nom de l'événement	BillcycleCreated
Producteur	svc-facturation
Consommateurs	svc-audit
Topic / exchange	facturation.billcycle-created
Payload (schéma)	Champs: billcycleId (integer). <i>Exemple JSON:</i> { "billcycleId": 20260801 }
Version / compatibilité	v1
Déclenché par / réactions	Job planifié mensuel ou lot du système de facturation ; Déclenche la génération des factures récurrentes.
Champ	Contenu
Nom de l'événement	CustomerbillCreated
Producteur	svc-facturation
Consommateurs	svc-audit, svc-clients
Topic / exchange	facturation.customerbill-created
Payload (schéma)	Champs: customerbillId (integer), totalAmount (number).

	<i>Exemple JSON:</i> { "customerbillId": 5001, "totalAmount": 125.50 }
Version / compatibilité	v1
Déclenché par / réactions	Résultat d'un DoInvoicing ou BillcycleCreated ; Met à disposition la facture pour le client et attend le paiement.
Champ	Contenu
Nom de l'événement	PaymentProcessed
Producteur	svc-facturation
Consommateurs	svc-audit, svc-clients, svc-commandes, svc-orchestration
Topic / exchange	facturation.payment-processed
Payload (schéma)	Champs: customerbillId (integer), paidAmount (number), amountDue (number). <i>Exemple JSON:</i> { "customerbillId": 5001, "paidAmount": 125.50, "amountDue": 0.0 }
Version / compatibilité	v1
Déclenché par / réactions	Traitement d'un paiement via la passerelle de paiement ; Solde du compte à jour, clôture l'attente de paiement.
Champ	Contenu
Nom de l'événement	DetectionFraude
Producteur	svc-audit
Consommateurs	svc-clients, svc-orchestration
Topic / exchange	audit.alerte-fraude
Payload (schéma)	Champs: eventType (sim-swap, identite), eventTime, priority, MSISDN, partyRef. <i>Exemple JSON:</i> { "eventType": "sim-swap", "eventTime": "2026-08-05T09:00:00Z", "priority": "1", "MSISDN": "+15145551234", "partyRef": "PTY-456" }
Version / compatibilité	v1
Déclenché par / réactions	Algorithmes d'analyse ou rapports manuels signalant une fraude ; Entraîne des blocages proactifs de lignes/comptes.
Champ	Contenu
Nom de l'événement	AbusUsage
Producteur	svc-audit

Consommateurs	svc-clients, svc-lignes, svc-facturation
Topic / exchange	audit.abus-usage
Payload (schéma)	Champs: usageType (SMS, data, voice), eventTime, subscriberRef, MSISDN. <i>Exemple JSON:</i> <pre>{ "usageType": "data", "eventTime": "2026-08-05T09:00:00Z", "MSISDN": "+15145551234", "subscriberRef": "SUB-123" }</pre>
Version / compatibilité	v1
Déclenché par / réactions	Analyse des données d'appels (CDRs) montrant un dépassement sévère (fair-use) ; Entraîne un bridage du réseau et/ou avertissement client.
Champ	Contenu
Nom de l'événement	ServiceProvisionOk (avec serviceStatus de type portabilité)
Producteur	svc-lignes
Consommateurs	svc-orchestration
Topic / exchange	lignes.service-provision-ok
Payload (schéma)	Champs: orderId (integer), validation (OK, KO), service (object avec serviceStatus). <i>Exemple JSON:</i> <pre>{ "orderId": 1002, "validation": "OK", "service": { "userId": 123, "orderId": 1002, "MSISDN": "+15145551234", "serviceStatus": "porting_in" } }</pre>
Version / compatibilité	v1
Déclenché par / réactions	Les échanges avec le consortium de portabilité (NPAC Canada) ; Confirme la finalisation du transfert et l'activation du nouveau réseau.

Annexe B — Matrice des récits utilisateur (couverture complète)

Les huit cas d'utilisation du cahier de charge CanTelcoX (UC-01 à UC-08). Tous les cas d'utilisation ont été documentés en entier dans la phase I.

ID	Récit utilisateur (cahier)	Priorité MoSCoW	Statut	Preuve
UC-01	Inscription & vérification d'identité	Must	Complet	Démonstration lors de la présentation
UC-02	Authentification & MFA (OTP / WebAuthn)	Must	Partiel	OTP seul. pour opération sensible à implémenter
UC-03	Activation d'une ligne mobile (MSISDN / portabilité)	Must	Partiel	Portabilité à finaliser d'implémenter
UC-04	Consultation de l'usage et des factures	Must	Partiel	Consultation des usages à implémenter
UC-05	Prise de commande / souscription à un forfait	Must	Complet	Postman bout-en-bout
UC-06	Paiement de facture mobile	Must	Complet	Postman <i>Appliquer un paiement à une facture</i>
UC-07	Détection de fraude (SIM swap, usage anormal, roaming)	Must	À faire	Service d'audit à implémenter
UC-08	Cycle de facturation mensuel	Must	À faire	Générer des factures en lot à partir des usages à implémenter

Annexe C — Traçabilité, Définition de Fini & auto-évaluation

C.1 Matrice de traçabilité (livrables & critères d'acceptation)

Cochez chaque case (☐) lorsque le livrable est produit et le critère satisfait ; indiquez la preuve (figure, section ou chemin dans le dépôt). « Empl. » renvoie à la section du présent document.

Tâche (énoncé Phase 2)	Livrables → empl.	Critères d'acceptation → empl.	Preuve	Fait
User stories complets	§1.3 ; Annexe B	Tous couverts	Tous couverts avec référence PI	☐
Items Phase 1 repris	§1.4 ; Annexe C	Items comblés + tracés	Dette technique identifiée et expliquée	☐
Architecture événementielle	§3–§6	Outbox + saga opérationnels	Outbox et Saga événementielle conceptuelle avec début d'implémentation non fonctionnelle	☐
Outbox pattern	§4.3 ; §8.1	Publication atomique, 0 perte	Explication comportement <i>exactly-once</i> .	☐
Saga chorégraphiée & compensation	§4.4 ; §6	Nominal + compensation démontrés	Cas nominal et compensation conceptuelle avec début d'implémentation non fonctionnelle	☐
Observabilité + tracing	§7.2 ; §8.6 ; §10.4	Trace bout-en-bout + 4 GS	Traces et métriques collectées mais manquante les événements de la PII, implémentation non fonctionnelle	☐
Charge (k6/JMeter/Artillery)	§10.2–10.3	Paliers NFR vérifiés	Pas pu démontrer car implémentation non	☐

			fonctionnelle	
Systèmes fournis (HLR & PortabilityHub)	§2.1 ; §4.6 ; §6.1/6.4 ; §8.9	Intégration obligatoire démontrée	Intégrés dans le code et partiellement exploité (HLR seul). Portabilité conceptuelle.	☐
ADR event-driven	§9	ADR-004..008 complets	Présents	☐

C.2 Définition de Fini (DoD) — Phase 2

- ☐ Tous les récits utilisateur du cahier sont couverts (Annexe B).
- ☐ Les items manquants de la Phase 1 sont repris et tracés (§1.4).
- ☐ Architecture événementielle opérationnelle : outbox pattern + saga chorégraphiée avec compensation.
- ☐ Consommateurs idempotents ; livraison at-least-once ; DLQ + retries en place.
- ☐ Journal d'audit append-only alimenté par les événements (CRTC/Loi 25).
- ☐ Observabilité : logs structurés, métriques Prometheus, tracing distribué, dashboards Grafana (4 Golden Signals).
- ☐ Paliers NFR event-driven atteints ou écarts argumentés ($P95 \leq 250$ ms, ≥ 1000 ops/s, dispo 99 %) ; comparatif sync ↔ événementiel.
- ☐ Intégration effective des systèmes fournis HLR/HSS et PortabilityHub (adapters/ACL, idempotence, compensation sur échec) — pas de simulation maison.
- ☐ API Gateway, load balancing et caching reconduits (Renvoi Phase 1).
- ☐ ≥ 5 ADR Phase 2 approuvés (ADR-004 à ADR-008) ; Arc42 + 4+1 cohérents ; CI/CD fonctionnels.
- ☐ docker compose up lance l'ensemble (services + broker + observabilité) ; reproductibilité < 30 min.
- ☐ Rapport PDF + dépôt projet (zip : code, scripts, dashboards, doc) remis sur Moodle.

C.3 Auto-évaluation

Critère	Section(s) de preuve	Auto-éval
Complétion user stories & items Phase 1	§1.3–1.4 ; Annexes B, C	[À compléter]
Architecture événementielle (outbox + saga)	§3–§9	[À compléter]
Observabilité, tracing & performance	§7 ; §8.6 ; §10	[À compléter]
Documentation (Arc42, 4+1, ADR)	Tout le document	[À compléter]

Annexe D — Soutenance et démonstration

Durée : 12 à 15 minutes de présentation + questions. Objectif : démontrer que l'architecture événementielle fonctionne (saga nominale + compensation) et que les décisions sont défendables.

D.1 Déroulé attendu

Étape	Contenu	Durée
Contexte & objectifs	Introduction et schémas de déploiement	3 minutes
Architecture événementielle	Hexagonale	2 minutes
Démo saga nominale	Flux de bout en bout + tracing	4 minutes
Démo compensation & résilience	-	-
Mesures & gains	-	-
Questions	—	6 minutes

D.2 Scénario de démonstration

Élément	Détail
Flux démontré	Inscription > génération OTP > Identification et authentification > Consultation du profil > Création d'une commande > Consultation de la commande > Consultation de la facture > Paiement de la facture
Cas de compensation	– non implémenté
Preuve d'idempotence	– Clé d'idempotence dans la charge utile soumise au système – Rejou bloqué
Observabilité montrée	Traces du système dans Jaeger et métriques dans Grafana

Annexe E - AsyncAPI

```
asyncapi: 3.1.0
id: 'urn:ets:log430:e26:g2:e6:cantelcoxapi:1.0'
info:
  title: CanTelcoX - Événements Kafka (production)
  version: 1.0.0
  description: |
    Contrat **AsyncAPI** de CanTelcoX – version événementielle (production).
  license:
    name: Apache 2.0
    url: 'https://www.apache.org/licenses/LICENSE-2.0'

defaultContentType: application/json

servers:
  kafka:
    host: 'kafka:9092'
    protocol: kafka
    description: Grappe Kafka de production
    bindings:
      kafka:
        bindingVersion: '0.5.0'

channels:
  alerteFraude:
    address: cantelcox.audit.1.0.event.alerte.fraude
    description: Avis de détection d'une fraude
    tags:
      - name: domaine
    bindings:
      kafka:
        topic: audit.alerte-fraude
        bindingVersion: '0.5.0'
    messages:
      DetectionFraude:
        $ref: '#/components/messages/DetectionFraude'
  abusUsage:
    address: cantelcox.audit.1.0.event.abus.usage
    description: Avis d'abus d'usage de service
    tags:
      - name: domaine
    bindings:
      kafka:
        topic: audit.abus-usage
        bindingVersion: '0.5.0'
    messages:
      AbusUsage:
        $ref: '#/components/messages/AbusUsage'
  offeringsVerified:
    address: cantelcox.catalogue.1.0.event.offerings.verified
    description: Avis que les offres commerciales sont vérifiées
    tags:
      - name: domaine
    bindings:
      kafka:
        topic: catalogue.offerings-verified
        bindingVersion: '0.5.0'
    messages:
      OfferingsVerified:
        $ref: '#/components/messages/OfferingsVerified'
  offeringsInvalid:
    address: cantelcox.catalogue.1.0.event.offerings.invalid
    description: Avis que les offres commerciales sont invalides
    tags:
      - name: compensation
    bindings:
      kafka:
        topic: catalogue.offerings-invalid
        bindingVersion: '0.5.0'
```

```

    messages:
      OfferingsInvalid:
        $ref: '#/components/messages/OfferingsInvalid'
stockDecreased:
  address: cantelcox.catalogue.1.0.event.stock.decreased
  description: Avis que le stock a été réduit
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: catalogue.stock-decreased
      bindingVersion: '0.5.0'
  messages:
    StockDecreased:
      $ref: '#/components/messages/StockDecreased'
stockIncreased:
  address: cantelcox.catalogue.1.0.event.stock.increased
  description: Avis que le stock a été augmenté
  tags:
    - name: compensation
  bindings:
    kafka:
      topic: catalogue.stock-increased
      bindingVersion: '0.5.0'
  messages:
    StockIncreased:
      $ref: '#/components/messages/StockIncreased'
orderSagaSubmitted:
  address: cantelcox.saga.1.0.event.order.submitted
  description: Avis que la saga de création de commande est soumise
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: saga.order-submitted
      bindingVersion: '0.5.0'
  messages:
    OrderSagaSubmitted:
      $ref: '#/components/messages/OrderSagaSubmitted'
orderValidationsOk:
  address: cantelcox.saga.1.0.event.order.validations.ok
  description: Avis que les validations d'une saga de commande sont correctes
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: saga.order-validations-ok
      bindingVersion: '0.5.0'
  messages:
    OrderValidationsOk:
      $ref: '#/components/messages/OrderValidationsOk'
orderValidationsKo:
  address: cantelcox.saga.1.0.event.order.validations.ko
  description: Avis que les validations d'une saga de commande sont incorrectes
  tags:
    - name: compensation
  bindings:
    kafka:
      topic: saga.order-validations-ko
      bindingVersion: '0.5.0'
  messages:
    OrderValidationsKo:
      $ref: '#/components/messages/OrderValidationsKo'
doProvisionActions:
  address: cantelcox.saga.1.0.action.provision.do
  description: Avis d'exécution les actions de provisionner (activer) les services d'une
commande
  tags:
    - name: domaine
  bindings:
    kafka:

```

```

      topic: saga.order-do-provision
      bindingVersion: '0.5.0'
    messages:
      DoProvisionActions:
        $ref: '#/components/messages/DoProvisionActions'
  doInvoicing:
    address: cantelcox.saga.1.0.action.invoicing.do
    description: Avis d'exécution de l'action de facturer la commande
    tags:
      - name: domaine
    bindings:
      kafka:
        topic: saga.order-do-invoicing
        bindingVersion: '0.5.0'
    messages:
      DoInvoicing:
        $ref: '#/components/messages/DoInvoicing'
  orderSagaEnded:
    address: cantelcox.saga.1.0.event.saga.order.ended
    description: Avis que la saga de commande est terminée
    tags:
      - name: domaine
    bindings:
      kafka:
        topic: saga.order-ended
        bindingVersion: '0.5.0'
    messages:
      OrderSagaEnded:
        $ref: '#/components/messages/OrderSagaEnded'
  buyerVerified:
    address: cantelcox.client.1.0.event.buyer.verified
    description: Avis que le client a été vérifié avec succès
    tags:
      - name: domaine
    bindings:
      kafka:
        topic: client.buyer-verified
        bindingVersion: '0.5.0'
    messages:
      BuyerVerified:
        $ref: '#/components/messages/BuyerVerified'
  buyerInvalid:
    address: cantelcox.client.1.0.event.buyer.invalid
    description: Avis que la validation du client est incorrecte
    tags:
      - name: compensation
    bindings:
      kafka:
        topic: client.buyer-invalid
        bindingVersion: '0.5.0'
    messages:
      BuyerInvalid:
        $ref: '#/components/messages/BuyerInvalid'
  partyIdentified:
    address: cantelcox.client.1.0.event.party.identified
    description: Avis que l'abonné a été identifié
    tags:
      - name: domaine
    bindings:
      kafka:
        topic: client.party-identified
        bindingVersion: '0.5.0'
    messages:
      PartyIdentified:
        $ref: '#/components/messages/PartyIdentified'
  orderDraft:
    address: cantelcox.commande.1.0.event.order.draft
    description: Avis que la commande est à l'état brouillon
    tags:
      - name: domaine
    bindings:

```

```

    kafka:
      topic: commande.order-draft
      bindingVersion: '0.5.0'
  messages:
    OrderDraft:
      $ref: '#/components/messages/OrderDraft'
orderCreated:
  address: cantelcox.commande.1.0.event.order.created
  description: Avis que la commande est à l'état créée
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: commande.order-created
      bindingVersion: '0.5.0'
  messages:
    OrderCreated:
      $ref: '#/components/messages/OrderCreated'
orderCancelled:
  address: cantelcox.commande.1.0.event.order.cancelled
  description: Avis que la commande est à l'état annulée
  tags:
    - name: compensation
  bindings:
    kafka:
      topic: commande.order-cancelled
      bindingVersion: '0.5.0'
  messages:
    OrderCancelled:
      $ref: '#/components/messages/OrderCancelled'
billcycleCreated:
  address: cantelcox.facturation.1.0.event.billcycle.created
  description: Avis que le cycle de facturation est créé
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: facturation.billcycle-created
      bindingVersion: '0.5.0'
  messages:
    BillcycleCreated:
      $ref: '#/components/messages/BillcycleCreated'
customerbillCreated:
  address: cantelcox.facturation.1.0.event.customerbill.created
  description: Avis que la facture client est créée
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: facturation.customerbill-created
      bindingVersion: '0.5.0'
  messages:
    CustomerbillCreated:
      $ref: '#/components/messages/CustomerbillCreated'
paymentProcessed:
  address: cantelcox.facturation.1.0.event.payment.processed
  description: Avis qu'un paiement a été traité
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: facturation.payment-processed
      bindingVersion: '0.5.0'
  messages:
    PaymentProcessed:
      $ref: '#/components/messages/PaymentProcessed'
paymentDelinquant:
  address: cantelcox.facturation.1.0.event.payment.delinquant
  description: Avis qu'un paiement n'a pas été traité et que le client est en délinquance
  tags:
    - name: compensation

```

```

bindings:
  kafka:
    topic: facturation.payment-delinquant
    bindingVersion: '0.5.0'
  messages:
    PaymentDelinquant:
      $ref: '#/components/messages/PaymentDelinquant'
serviceVerified:
  address: cantelcox.lignes.1.0.event.service.verified
  description: Avis qu'une élligibilité de service est vérifiée
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: lignes.service-verified
      bindingVersion: '0.5.0'
  messages:
    ServiceVerified:
      $ref: '#/components/messages/ServiceVerified'
serviceProvisionOk:
  address: cantelcox.lignes.1.0.event.service.provision.ok
  description: Avis qu'une activation de service est complétée avec succès
  tags:
    - name: domaine
  bindings:
    kafka:
      topic: lignes.service-provision-ok
      bindingVersion: '0.5.0'
  messages:
    ServiceProvisionOk:
      $ref: '#/components/messages/ServiceProvisionOk'
serviceProvisionKo:
  address: cantelcox.lignes.1.0.event.service.provision.ko
  description: Avis qu'une activation de service n'a pas réussie
  tags:
    - name: compensation
  bindings:
    kafka:
      topic: lignes.service-provision-ko
      bindingVersion: '0.5.0'
  messages:
    ServiceProvisionKo:
      $ref: '#/components/messages/ServiceProvisionKo'

operations:

popAlerteFraude:
  action: receive
  channel:
    $ref: '#/channels/alerteFraude'
  summary: >-
    Réagir quand une fraude est détectée.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-clients
          - svc-orchestration
  messages:
    - $ref: '#/channels/alerteFraude/messages/DetectionFraude'
pushAlerteFraude:
  action: send
  channel:
    $ref: '#/channels/alerteFraude'
  summary: >-
    Envoyer une alerte de fraude.
  bindings:
    kafka:
      clientId:
        type: string

```

```

        enum:
          - svc-audit
      messages:
        - $ref: '#/channels/alerteFraude/messages/DetectionFraude'
    popAbusUsage:
      action: receive
      channel:
        $ref: '#/channels/abusUsage'
      summary: >-
        Réagir quand un abus d'usage est détecté.
      bindings:
        kafka:
          clientId:
            type: string
            enum:
              - svc-clients
              - svc-lignes
              - svc-facturation
      messages:
        - $ref: '#/channels/abusUsage/messages/AbusUsage'
    pushAbusUsage:
      action: send
      channel:
        $ref: '#/channels/abusUsage'
      summary: >-
        Envoyer un avis d'abus d'usage.
      bindings:
        kafka:
          clientId:
            type: string
            enum:
              - svc-audit
      messages:
        - $ref: '#/channels/abusUsage/messages/AbusUsage'
    popOfferingsVerified:
      action: receive
      channel:
        $ref: '#/channels/offeringsVerified'
      summary: >-
        Réagir quand les offres sont vérifiées.
      bindings:
        kafka:
          clientId:
            type: string
            enum:
              - svc-orchestration
      messages:
        - $ref: '#/channels/offeringsVerified/messages/OfferingsVerified'
    pushOfferingsVerified:
      action: send
      channel:
        $ref: '#/channels/offeringsVerified'
      summary: >-
        Envoyer une confirmation de vérification des offres.
      bindings:
        kafka:
          clientId:
            type: string
            enum:
              - svc-catalogue
      messages:
        - $ref: '#/channels/offeringsVerified/messages/OfferingsVerified'
    popOfferingsInvalid:
      action: receive
      channel:
        $ref: '#/channels/offeringsInvalid'
      summary: >-
        Réagir quand les offres sont invalides.
      bindings:
        kafka:
          clientId:

```

```

        type: string
        enum:
          - svc-orchestration
      messages:
        - $ref: '#/channels/offeringsInvalid/messages/OfferingsInvalid'
pushOfferingsInvalid:
  action: send
  channel:
    $ref: '#/channels/offeringsInvalid'
  summary: >-
    Envoyer une notification d'offres invalides.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-catalogue
      messages:
        - $ref: '#/channels/offeringsInvalid/messages/OfferingsInvalid'
popStockDecreased:
  action: receive
  channel:
    $ref: '#/channels/stockDecreased'
  summary: >-
    Réagir quand les stocks sont déduits.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-orchestration
      messages:
        - $ref: '#/channels/stockDecreased/messages/StockDecreased'
pushStockDecreased:
  action: send
  channel:
    $ref: '#/channels/stockDecreased'
  summary: >-
    Envoyer une notification de réduction de stock.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-catalogue
      messages:
        - $ref: '#/channels/stockDecreased/messages/StockDecreased'
popStockIncreased:
  action: receive
  channel:
    $ref: '#/channels/stockIncreased'
  summary: >-
    Réagir quand les stocks sont augmentés suite à une compensation.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-orchestration
      messages:
        - $ref: '#/channels/stockIncreased/messages/StockIncreased'
pushStockIncreased:
  action: send
  channel:
    $ref: '#/channels/stockIncreased'
  summary: >-
    Envoyer une notification d'augmentation de stock.
  bindings:
    kafka:
      clientId:
        type: string

```

```

        enum:
        - svc-catalogue
    messages:
    - $ref: '#/channels/stockIncreased/messages/StockIncreased'
popOrderSagaSubmitted:
  action: receive
  channel:
    $ref: '#/channels/orderSagaSubmitted'
  summary: >-
    Réagir quand la saga de commande est soumise.
  bindings:
    kafka:
      clientId:
        type: string
      enum:
      - svc-audit
      - svc-commandes
    messages:
    - $ref: '#/channels/orderSagaSubmitted/messages/OrderSagaSubmitted'
pushOrderSagaSubmitted:
  action: send
  channel:
    $ref: '#/channels/orderSagaSubmitted'
  summary: >-
    Envoyer une notification de soumission de saga de commande.
  bindings:
    kafka:
      clientId:
        type: string
      enum:
      - svc-orchestration
    messages:
    - $ref: '#/channels/orderSagaSubmitted/messages/OrderSagaSubmitted'
popOrderValidationsOk:
  action: receive
  channel:
    $ref: '#/channels/orderValidationsOk'
  summary: >-
    Réagir quand la validation de la commande est bonne.
  bindings:
    kafka:
      clientId:
        type: string
      enum:
      - svc-commandes
    messages:
    - $ref: '#/channels/orderValidationsOk/messages/OrderValidationsOk'
pushOrderValidationsOk:
  action: send
  channel:
    $ref: '#/channels/orderValidationsOk'
  summary: >-
    Envoyer une confirmation de validation réussie de commande.
  bindings:
    kafka:
      clientId:
        type: string
      enum:
      - svc-orchestration
    messages:
    - $ref: '#/channels/orderValidationsOk/messages/OrderValidationsOk'
popOrderValidationsKo:
  action: receive
  channel:
    $ref: '#/channels/orderValidationsKo'
  summary: >-
    Réagir quand la validation de la commande est incorrecte.
  bindings:
    kafka:
      clientId:
        type: string

```



```

        enum:
          - svc-audit
          - svc-clients
      messages:
        - $ref: '#/channels/orderValidationsKo/messages/OrderValidationsKo'
pushOrderValidationsKo:
  action: send
  channel:
    $ref: '#/channels/orderValidationsKo'
  summary: >-
    Envoyer une notification de validation échouée de commande.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-orchestration
      messages:
        - $ref: '#/channels/orderValidationsKo/messages/OrderValidationsKo'
popDoProvisionActions:
  action: receive
  channel:
    $ref: '#/channels/doProvisionActions'
  summary: >-
    Réagir à une demande d'activation de services.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-lignes
      messages:
        - $ref: '#/channels/doProvisionActions/messages/DoProvisionActions'
pushDoProvisionActions:
  action: send
  channel:
    $ref: '#/channels/doProvisionActions'
  summary: >-
    Demander l'activation de services.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-orchestration
      messages:
        - $ref: '#/channels/doProvisionActions/messages/DoProvisionActions'
popDoInvoicing:
  action: receive
  channel:
    $ref: '#/channels/doInvoicing'
  summary: >-
    Réagir à une demande de génération de facturation.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-facturation
      messages:
        - $ref: '#/channels/doInvoicing/messages/DoInvoicing'
pushDoInvoicing:
  action: send
  channel:
    $ref: '#/channels/doInvoicing'
  summary: >-
    Demander la génération de la facturation en lot périodique.
  bindings:
    kafka:

```

```

      clientId:
        type: string
        enum:
          - svc-orchestration
    messages:
      - $ref: '#/channels/doInvoicing/messages/DoInvoicing'
  popOrderSagaEnded:
    action: receive
    channel:
      $ref: '#/channels/orderSagaEnded'
    summary: >-
      Réagir quand la saga de commande est terminée
    bindings:
      kafka:
        clientId:
          type: string
          enum:
            - svc-audit
            - svc-clients
    messages:
      - $ref: '#/channels/orderSagaEnded/messages/OrderSagaEnded'
  pushOrderSagaEnded:
    action: send
    channel:
      $ref: '#/channels/orderSagaEnded'
    summary: >-
      Envoyer une notification de fin de saga de commande.
    bindings:
      kafka:
        clientId:
          type: string
          enum:
            - svc-orchestration
    messages:
      - $ref: '#/channels/orderSagaEnded/messages/OrderSagaEnded'
  popBuyerVerified:
    action: receive
    channel:
      $ref: '#/channels/buyerVerified'
    summary: >-
      Réagir quand le client est vérifié
    bindings:
      kafka:
        clientId:
          type: string
          enum:
            - svc-orchestration
    messages:
      - $ref: '#/channels/buyerVerified/messages/BuyerVerified'
  pushBuyerVerified:
    action: send
    channel:
      $ref: '#/channels/buyerVerified'
    summary: >-
      Envoyer une confirmation de vérification du client.
    bindings:
      kafka:
        clientId:
          type: string
          enum:
            - svc-clients
    messages:
      - $ref: '#/channels/buyerVerified/messages/BuyerVerified'
  popBuyerInvalid:
    action: receive
    channel:
      $ref: '#/channels/buyerInvalid'
    summary: >-
      Réagir quand la vérification du client est invalide
    bindings:
      kafka:

```

```

      clientId:
        type: string
        enum:
          - svc-orchestration
    messages:
      - $ref: '#/channels/buyerInvalid/messages/BuyerInvalid'
pushBuyerInvalid:
  action: send
  channel:
    $ref: '#/channels/buyerInvalid'
  summary: >-
    Envoyer une notification de vérification invalide du client.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-clients
    messages:
      - $ref: '#/channels/buyerInvalid/messages/BuyerInvalid'
popPartyIdentified:
  action: receive
  channel:
    $ref: '#/channels/partyIdentified'
  summary: >-
    Réagir quand un abonné est identifié
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
    messages:
      - $ref: '#/channels/partyIdentified/messages/PartyIdentified'
pushPartyIdentified:
  action: send
  channel:
    $ref: '#/channels/partyIdentified'
  summary: >-
    Envoyer une notification d'identification d'abonné.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-clients
    messages:
      - $ref: '#/channels/partyIdentified/messages/PartyIdentified'
popOrderDraft:
  action: receive
  channel:
    $ref: '#/channels/orderDraft'
  summary: >-
    Réagir quand une commande est en état brouillon (procéder aux vérifications)
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-clients
          - svc-catalogue
          - svc-ligne
    messages:
      - $ref: '#/channels/orderDraft/messages/OrderDraft'
pushOrderDraft:
  action: send
  channel:
    $ref: '#/channels/orderDraft'
  summary: >-
    Envoyer une notification de commande en état brouillon.
  bindings:

```

```

    kafka:
      clientId:
        type: string
        enum:
          - svc-commandes
    messages:
      - $ref: '#/channels/orderDraft/messages/OrderDraft'
popOrderCreated:
  action: receive
  channel:
    $ref: '#/channels/orderCreated'
  summary: >-
    Réagir quand une commande est en état créée
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-catalogue
          - svc-orchestration
    messages:
      - $ref: '#/channels/orderCreated/messages/OrderCreated'
pushOrderCreated:
  action: send
  channel:
    $ref: '#/channels/orderCreated'
  summary: >-
    Envoyer une notification de commande créée.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-commandes
    messages:
      - $ref: '#/channels/orderCreated/messages/OrderCreated'
popOrderCancelled:
  action: receive
  channel:
    $ref: '#/channels/orderCancelled'
  summary: >-
    Réagir quand une commande est annulée (compensation)
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-catalogue
          - svc-orchestration
          - svc-clients
    messages:
      - $ref: '#/channels/orderCancelled/messages/OrderCancelled'
pushOrderCancelled:
  action: send
  channel:
    $ref: '#/channels/orderCancelled'
  summary: >-
    Envoyer une notification d'annulation de commande.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-commandes
    messages:
      - $ref: '#/channels/orderCancelled/messages/OrderCancelled'
popBillcycleCreated:
  action: receive
  channel:

```

```

    $ref: '#/channels/billcycleCreated'
summary: >-
  Réagir quand un cycle de facturation est créé
bindings:
  kafka:
    clientId:
      type: string
      enum:
        - svc-audit
  messages:
    - $ref: '#/channels/billcycleCreated/messages/BillcycleCreated'
pushBillcycleCreated:
  action: send
  channel:
    $ref: '#/channels/billcycleCreated'
summary: >-
  Envoyer une notification de création de cycle de facturation.
bindings:
  kafka:
    clientId:
      type: string
      enum:
        - svc-facturation
  messages:
    - $ref: '#/channels/billcycleCreated/messages/BillcycleCreated'
popCustomerbillCreated:
  action: receive
  channel:
    $ref: '#/channels/customerbillCreated'
summary: >-
  Réagir quand une facture client est créée
bindings:
  kafka:
    clientId:
      type: string
      enum:
        - svc-audit
        - svc-clients
  messages:
    - $ref: '#/channels/customerbillCreated/messages/CustomerbillCreated'
pushCustomerbillCreated:
  action: send
  channel:
    $ref: '#/channels/customerbillCreated'
summary: >-
  Envoyer une notification de création de facture client.
bindings:
  kafka:
    clientId:
      type: string
      enum:
        - svc-facturation
  messages:
    - $ref: '#/channels/customerbillCreated/messages/CustomerbillCreated'
popPaymentProcessed:
  action: receive
  channel:
    $ref: '#/channels/paymentProcessed'
summary: >-
  Réagir quand un paiement est traité
bindings:
  kafka:
    clientId:
      type: string
      enum:
        - svc-audit
        - svc-clients
  messages:
    - $ref: '#/channels/paymentProcessed/messages/PaymentProcessed'
pushPaymentProcessed:
  action: send

```

```

channel:
  $ref: '#/channels/paymentProcessed'
summary: >-
  Envoyer une notification de paiement traité.
bindings:
  kafka:
    clientId:
      type: string
      enum:
        - svc-facturation
    messages:
      - $ref: '#/channels/paymentProcessed/messages/PaymentProcessed'
popPaymentDelinquant:
  action: receive
  channel:
    $ref: '#/channels/paymentDelinquant'
  summary: >-
    Réagir quand un paiement devient délinquant
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-clients
          - svc-ligne
          - svc-commandes
      messages:
        - $ref: '#/channels/paymentDelinquant/messages/PaymentDelinquant'
pushPaymentDelinquant:
  action: send
  channel:
    $ref: '#/channels/paymentDelinquant'
  summary: >-
    Envoyer une notification de paiement délinquant.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-facturation
      messages:
        - $ref: '#/channels/paymentDelinquant/messages/PaymentDelinquant'
popServiceVerified:
  action: receive
  channel:
    $ref: '#/channels/serviceVerified'
  summary: >-
    Réagir quand un service est vérifié
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-orchestration
      messages:
        - $ref: '#/channels/serviceVerified/messages/ServiceVerified'
pushServiceVerified:
  action: send
  channel:
    $ref: '#/channels/serviceVerified'
  summary: >-
    Envoyer une confirmation de vérification de service.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-lignes
      messages:
        - $ref: '#/channels/serviceVerified/messages/ServiceVerified'

```

```

popServiceProvisionOk:
  action: receive
  channel:
    $ref: '#/channels/serviceProvisionOk'
  summary: >-
    Réagir quand un service est activé avec succès
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-orchestration
      messages:
        - $ref: '#/channels/serviceProvisionOk/messages/ServiceProvisionOk'
pushServiceProvisionOk:
  action: send
  channel:
    $ref: '#/channels/serviceProvisionOk'
  summary: >-
    Envoyer une confirmation d'activation réussie de service.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-lignes
      messages:
        - $ref: '#/channels/serviceProvisionOk/messages/ServiceProvisionOk'
popServiceProvisionKo:
  action: receive
  channel:
    $ref: '#/channels/serviceProvisionKo'
  summary: >-
    Réagir quand l'activation d'un service a échouée
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-clients
          - svc-orchestration
      messages:
        - $ref: '#/channels/serviceProvisionKo/messages/ServiceProvisionKo'
pushServiceProvisionKo:
  action: send
  channel:
    $ref: '#/channels/serviceProvisionKo'
  summary: >-
    Envoyer une notification d'échec d'activation de service.
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-lignes
      messages:
        - $ref: '#/channels/serviceProvisionKo/messages/ServiceProvisionKo'

```

components:

```

correlationIds:
  default:
    description: Identifiant métier de corrélation (clé d'idempotence)
    location: $message.header#/correlationId

messages:
  DetectionFraude:
    name: DetectionFraude
    title: detection-fraude

```

```

summary: >-
  Informe sur l'occurrence d'une fraude
contentType: application/json
traits:
  - $ref: '#/components/messageTraits/commonHeaders'
payload:
  $ref: '#/components/schemas/DetectionFraudePayload'
AbusUsage:
  name: AbusUsage
  title: abus-usage
  summary: >-
    Informe sur l'occurrence d'un abus d'usage de service
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/AbusUsagePayload'
OfferingsVerified:
  name: OfferingsVerified
  title: offerings-verified
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/OfferingsVerifiedPayload'
OfferingsInvalid:
  name: OfferingsInvalid
  title: offerings-invalid
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/OfferingsInvalidPayload'
StockDecreased:
  name: StockDecreased
  title: stock-decreased
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/StockPayload'
StockIncreased:
  name: StockIncreased
  title: stock-increased
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/StockPayload'
OrderSagaSubmitted:
  name: OrderSagaSubmitted
  title: order-saga-submitted
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/OrderSagaSubmittedPayload'
OrderValidationsOk:
  name: OrderValidationsOk
  title: order-validations-ok
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/OrderValidationsPayload'
OrderValidationsKo:
  name: OrderValidationsKo
  title: order-validations-ko
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'

```



```

    payload:
      $ref: '#/components/schemas/OrderValidationsPayload'
  DoProvisionActions:
    name: DoProvisionActions
    title: do-provision-actions
    contentType: application/json
    traits:
      - $ref: '#/components/messageTraits/commonHeaders'
    payload:
      $ref: '#/components/schemas/DoProvisionActionsPayload'
  DoInvoicing:
    name: DoInvoicing
    title: do-invoicing
    contentType: application/json
    traits:
      - $ref: '#/components/messageTraits/commonHeaders'
    payload:
      $ref: '#/components/schemas/DoInvoicingPayload'
  OrderSagaEnded:
    name: OrderSagaEnded
    title: order-saga-ended
    contentType: application/json
    traits:
      - $ref: '#/components/messageTraits/commonHeaders'
    payload:
      $ref: '#/components/schemas/OrderSagaEndedPayload'
  BuyerVerified:
    name: BuyerVerified
    title: buyer-verified
    contentType: application/json
    traits:
      - $ref: '#/components/messageTraits/commonHeaders'
    payload:
      $ref: '#/components/schemas/BuyerValidationPayload'
  BuyerInvalid:
    name: BuyerInvalid
    title: buyer-invalid
    contentType: application/json
    traits:
      - $ref: '#/components/messageTraits/commonHeaders'
    payload:
      $ref: '#/components/schemas/BuyerValidationPayload'
  PartyIdentified:
    name: PartyIdentified
    title: party-identified
    contentType: application/json
    traits:
      - $ref: '#/components/messageTraits/commonHeaders'
    payload:
      $ref: '#/components/schemas/PartyIdentifiedPayload'
  OrderDraft:
    name: OrderDraft
    title: order-draft
    contentType: application/json
    traits:
      - $ref: '#/components/messageTraits/commonHeaders'
    payload:
      $ref: '#/components/schemas/OrderStatePayload'
  OrderCreated:
    name: OrderCreated
    title: order-created
    contentType: application/json
    traits:
      - $ref: '#/components/messageTraits/commonHeaders'
    payload:
      $ref: '#/components/schemas/OrderStatePayload'
  OrderCancelled:
    name: OrderCanceled
    title: order-cancelled
    contentType: application/json
    traits:

```

```

    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/OrderStatePayload'
BillcycleCreated:
  name: BillcycleCreated
  title: billcycle-created
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/BillcycleCreatedPayload'
CustomerbillCreated:
  name: CustomerbillCreated
  title: customerbill-created
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/CustomerbillCreatedPayload'
PaymentProcessed:
  name: PaymentProcessed
  title: payment-processed
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/PaymentProcessedPayload'
PaymentDelinquant:
  name: PaymentDelinquant
  title: payment-delinquant
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/PaymentDelinquantPayload'
ServiceVerified:
  name: ServiceVerified
  title: service-verified
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/ServiceVerifiedPayload'
ServiceProvisionOk:
  name: ServiceProvisionOk
  title: service-provision-ok
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/ServiceProvisionPayload'
ServiceProvisionKo:
  name: ServiceProvisionKo
  title: service-provision-ko
  contentType: application/json
  traits:
    - $ref: '#/components/messageTraits/commonHeaders'
  payload:
    $ref: '#/components/schemas/ServiceProvisionPayload'

schemas:
  DetectionFraudePayload:
    type: object
    description: Audit Event API v1
    required: [eventType, eventTime, correlationId, domain, event, party]
    properties:
      eventType:
        type: string
        enum: [sim-swap, identite]
        description: Type de fraude.
      eventTime:

```

```

    type: string
    format: date-time
  description:
    type: string
  priority:
    type: string
    example: "2"
  timeOccurred:
    type: string
    format: date-time
  partyRef:
    type: string
  MSISDN:
    type: string
AbusUsagePayload:
  type: object
  description: Audit Event API v1
  required: [usageType, eventTime, correlationId, subscriberRef, MSISDN]
  properties:
    usageType:
      type: string
      enum: [SMS, data, voice]
      description: Type d'abus d'usage
    eventTime:
      type: string
      format: date-time
    subscriberRef:
      type: string
    MSISDN:
      type: string
    timeOccurred:
      type: string
      format: date-time
OfferingsVerifiedPayload:
  type: object
  required: [orderId, productOfferings]
  properties:
    productOfferings:
      type: object
      description: Offres validées avec leur détail
      required: [name, code, offeringType]
      properties:
        name:
          type: string
        code:
          type: string
        description:
          type: string
        offeringType:
          enum: [produit, service]
    orderId:
      type: integer
      description: Référence à la commande
OfferingsInvalidPayload:
  type: object
  required: [orderId]
  properties:
    invalidproductofferings:
      type: array
      description: Codes des offres invalides
    orderId:
      type: integer
      description: Référence à la commande
StockPayload:
  type: object
  required: [orderId]
  properties:
    affectedProductOfferings:
      type: object
      description: Structure d'offres avec les quantités restantes de stock
      properties:

```

```

        productOfferingRef:
          type: string
        quantity_after:
          type: integer
      orderId:
        type: integer
        description: Référence à la commande
    OrderSagaSubmittedPayload:
      type: object
      required: [userId]
      properties:
        timeSubmitted:
          type: string
          format: date-time
      order:
        $ref: '#/components/schemas/OrderPayload'
    OrderValidationsPayload:
      type: object
      required: [orderId]
      properties:
        orderId:
          type: integer
        validation:
          type: string
          enum: [OK, KO]
    DoProvisionActionsPayload:
      type: object
      required: [provisioningId, orderId, userId, productOfferingRef, MSISDN]
      properties:
        provisioningId:
          type: string
          format: uuid
          description: Identifiant unique de cette tentative de provisionnement
        orderId:
          type: integer
        userId:
          type: integer
        productOfferingRef:
          type: string
          description: Référence de l'offre associée à la ligne
        MSISDN:
          type: string
          description: Numéro mobile du service
    DoInvoicingPayload:
      type: object
      required: [orderId, billCycleType]
      properties:
        orderId:
          type: integer
        billCycleType:
          type: string
          enum: [mensuel, ponctuel]
          description: Type de cycle de facturation
    OrderSagaEndedPayload:
      type: object
      required: [status, orderId]
      properties:
        status:
          type: string
        activation:
          type: object
          description: Détail de l'activation
          properties:
            lines:
              type: object
            status:
              type: string
        customerbillId:
          type: integer
        orderId:
          type: integer

```

```

    paymentLink:
      type: string
  BuyerValidationPayload:
    type: object
    required: [partyRef, orderId]
    properties:
      partyRef:
        type: string
      orderId:
        type: integer
      validation:
        type: string
        enum: [OK, KO]
  PartyIdentifiedPayload:
    type: object
    required: [partyRef]
    properties:
      partyRef:
        type: string
      identificationTime:
        type: string
        format: date-time
      validation:
        type: string
        enum: [OK, KO]
  OrderStatePayload:
    type: object
    required: [order, orderState]
    properties:
      orderState:
        type: string
        enum: [draft, created, cancelled]
      timeOccured:
        type: string
        format: date-time
      order:
        $ref: '#/components/schemas/OrderPayload'
  OrderPayload:
    type: object
    required: [userId, items]
    properties:
      userId:
        type: string
      items:
        type: object
        description: Items de la commande
  BillcycleCreatedPayload:
    type: object
    required: [billcycleId]
    properties:
      billcycleId:
        type: integer
  CustomerbillCreatedPayload:
    type: object
    required: [customerbillId]
    properties:
      customerbillId:
        type: integer
      totalAmount:
        type: number
  PaymentProcessedPayload:
    type: object
    required: [customerbillId]
    properties:
      customerbillId:
        type: integer
      paidAmount:
        type: number
      amountDue:
        type: number
  PaymentDelinquantPayload:

```

```

    type: object
    required: [customerbillId, userId]
    properties:
      userId:
        type: integer
      customerbillId:
        type: integer
      amountOverdue:
        type: number
  ServiceVerifiedPayload:
    type: object
    required: [serviceRef, orderId]
    properties:
      orderId:
        type: integer
      serviceRef:
        type: string
  ServiceProvisionPayload:
    type: object
    required: [provisioningId, productOfferingRef, orderId, validation, service]
    properties:
      provisioningId:
        type: string
        format: uuid
        description: Identifiant repris de DoProvisionActions
      productOfferingRef:
        type: string
        description: Référence de l'offre associée à la ligne
      service:
        $ref: '#/components/schemas/ServicePayload'
      orderId:
        type: integer
      validation:
        type: string
        enum: [OK, KO]
      failureReason:
        type: string
        description: Raison métier ou technique d'un résultat KO
  ServicePayload:
    type: object
    required: [userId, orderId, MSISDN, serviceStatus]
    properties:
      userId:
        type: integer
      orderId:
        type: integer
      MSISDN:
        type: string
      supi:
        type: string
      serviceStatus:
        type: string
        enum: [created, active, ported, porting_in, porting_out, canceled, error]

messageTraits:
  commonHeaders:
    headers:
      type: object
      required: [aggregate_type, aggregate_id]
      properties:
        aggregate_id:
          type: string
        aggregate_type:
          type: string

operationTraits:
  antifraude:
    bindings:
      kafka:
        clientId:
          type: string

```

```

        enum:
          - svc-clients
          - svc-orchestration
abususage:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-clients
          - svc-lignes
          - svc-facturation
saga:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-orchestration
auditcommande:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-commandes
auditclient:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-clients
auditligne:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-lignes
auditfacturation:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-facturation
audit:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
clientcatalogueligne:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-clients
          - svc-catalogue
          - svc-ligne
auditcataloguesaga:
  bindings:
    kafka:
      clientId:

```

```

    type: string
    enum:
      - svc-audit
      - svc-catalogue
      - svc-orchestration
auditcataloguesagaclient:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-catalogue
          - svc-orchestration
          - svc-clients
auditclientlignecommande:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-audit
          - svc-clients
          - svc-ligne
          - svc-commandes
commande:
  bindings:
    kafka:
      clientId:
        type: string
        enum:
          - svc-commandes

```